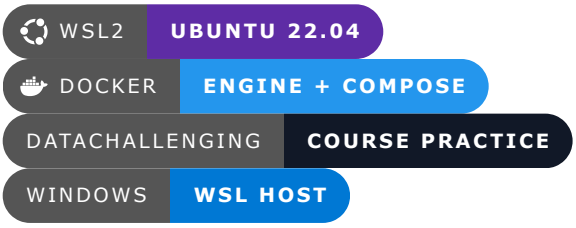
DataChallenging 手册

API 调用 • WSL2 • Ubuntu 22.04 LTS • Docker Engine • Docker Compose • DataChallenging



手册导航目录 📚

《大数据分析技术》DataChallenging 环境安装与实践手册

> **使用说明**：点击目录中的章节名称，可快速跳转至对应内容。首次安装环境的同学建议按顺序阅读；
>
> 已经完成环境安装的同学，可直接进入第三部分。

第一部分：课程概览与学习指南

- 文档速览：这份手册可以帮助你完成什么
- 推荐阅读顺序

第二部分：环境安装与配置

2.1 大模型 API 配置

- 华为云 API Key

2.2 WSL2 与 Ubuntu 环境安装

- 安装前提与系统要求
- 检查 WSL 与 Ubuntu 安装结果

2.3 Docker Engine 与 Docker Compose 安装

- 方案一：通过 Docker 官方软件源安装
- 方案二：通过 Ubuntu 仓库安装

2.4 启动 DataChallenging 本地环境

- 进入课程部署目录
- 检查 DataChallenging CLI

2.5 认识课程项目文件

- 明确工作区目录
- 明确失败模式
- 明确课程要求
- Task：一道完整的数据分析题 ⭐⭐⭐
- ⭐⭐⭐Agent：一道解决 Task 的程序
- ⭐⭐⭐元信息管理文件填写 ⭐⭐⭐
- 数据挖掘课程小组作业

第三部分：阶段1 — Task 与 Agent 初始设计

3.1 本地测试与提交阶段1作业

- 构建 Phase 1 提交包
- .env 应该如何配置?
- 验证最终提交 ZIP

3.2 理解 Agent 运行结果

- 情况一：Agent 正确完成任务
- 情况二：Agent 运行成功但答案错误
- 情况三：Agent 运行失败

第四部分：阶段 2 — 匿名任务挑战与优化

4.1 任务一：匿名Tasks分析BigData

- 导入匿名Tasks
- 测试匿名 Task
- 分析匿名 Task

- ⭐⭐⭐评审意见文件填写⭐⭐⭐

4.2 任务二：Task-Agent 最终方案

- 敲定Task 方案
- 敲定Agent 方案
- 测试 Task-Agent 最终方案
- 本地验证

4.3 任务三：实验报告内容要求

- 实验报告整理

第五部分：常见问题与提交前检查

5.1 常见问题排查

- Q1：Docker 服务没有启动怎么办？
- Q2：`docker compose` 不存在怎么办？
- Q3：`run` 失败怎么办？
- Q4：Agent 能运行，但是答案错误怎么办？

5.2 快速操作

- 系统推荐的实践流程

阅读建议

- **第一次使用**：第一部分 → 第二部分 → 第三部分。
- **已经安装好环境**：直接进入第三部分，开始 阶段 1。
- **收到匿名任务包**：直接进入第四部分，完成 阶段 2。

文档速览

模块	你要完成什么	成功标志
🧭 API 调用	登录华为云，查看额度，创建 API Key，确认 BaseURL	拿到模型名、额度、BaseURL

模块	你要完成什么	成功标志
🐧 WSL Ubuntu	在 Windows 上安装 WSL2 + Ubuntu 22.04 LTS	`wsl -l -v` 显示 `VERSION 2`
👑 root + systemd	以 root 进入 Ubuntu，并启用 systemd	`whoami=root`，`ps -p 1 -o comm=` 输出 `systemd`
🐳 Docker Engine	在 WSL Ubuntu 内安装 Docker，不走 Docker Desktop	`docker version` 显示 Client / Server
🧩 Docker Compose	安装并验证 Compose	`docker compose version` 正常输出
📊 DataChallenging	启动课程实践环境，创建 Task 与 Agent，并完成本地测试	`datac --help` 正常，Task/Agent 可运行

推荐阅读顺序

1. **API 调用**：先确认华为云登录、额度、API Key 和 BaseURL。
2. **WSL Ubuntu**：安装 Windows + WSL2 + Ubuntu 22.04 LTS。
3. **root + systemd**：设置 DataChallenging 所需的 Docker 运行环境。
4. **Docker Engine & Compose**：选择 Docker 官方源或 Ubuntu 仓库路线。
5. **DataChallenging 本地环境**：启动课程提供的本地服务，并确认 CLI 与浏览器入口可用。
6. **课程实践**：按 Task → Data → reference.py → Agent → Run → Build 的顺序完成阶段一，并进入匿名题目分析与迭代。

2.1 大模型 API 配置

获取 API Key

请按照“领取码道体验版 → 等待教师邀请加入班级 → 查看代金券到账情况”的顺序完成操作。(华为云开发者平台)

2.2 WSL2 与 Ubuntu 环境安装

安装前提与系统要求

🐧 ———————— Linux / WSL ———————— 🐧

Windows 安装 WSL2 → 在线安装 Ubuntu → 在 Ubuntu / WSL 内启用 systemd → 在线安装 Docker Engine + Docker Compose 插件 → 导入离线镜像 → 解压项目包 → 使用 Docker Compose 启动容器 → Windows 浏览器访问本地端口。

本手册采用“在 WSL Ubuntu 内安装 Docker Engine”的方式，不依赖 Docker Desktop。Microsoft 官方文档说明，`wsl --install` 可一键安装 WSL 并默认安装 Ubuntu，新安装的 Linux 发行版默认使用 WSL 2；Docker 官方文档推荐在 Ubuntu 中通过 Docker 的 apt 仓库安装 Docker Engine 与 `docker-compose-plugin`。

🐧 ———————— WSL Ubuntu ———————— 🐧

下面是**只安装 WSL2 + Ubuntu 22.04 LTS** 的完整流程。

这一步是在后续 Docker Engine / DataChallenging 课程实践之前完成的基础环境；

强调：wsl 命令一般在 Windows PowerShell 执行，

`apt`、`systemctl`、`docker` 等命令才是在 WSL Ubuntu 终端中执行 。

2.2.1 前提确认

适用系统：**Windows 11 或 Windows 10 版本 2004 及以上，Build 19041 及以上**。Microsoft 官方 WSL 安装文档说明，下面的 `wsl --install` 命令要求 Windows 10 version 2004+ / Build 19041+ 或 Windows 11。(Microsoft Learn)

建议先确认虚拟化已开启：

1. 打开任务管理器。
2. 进入 **性能 → CPU**。
3. 看右下角 **虚拟化** 是否为 **已启用**。

Ubuntu 官方 WSL 文档也提醒：WSL 需要启用虚拟化，`wsl --install` 会尝试启用 Virtual Machine Platform，但通常需要重启后生效。(Ubuntu Documentation)

---

a) 打开 Windows PowerShell 管理员模式

在 Windows 开始菜单搜索：

```
PowerShell
```

右键选择：

```
以管理员身份运行
```

后面的 `wsl ...` 命令都在这个 **Windows PowerShell 管理员窗口** 里执行。

---

b) 更新 WSL

先执行：

```
wsl --update
```

再查看 WSL 版本：

```
wsl --version
```

Microsoft WSL 基础命令文档说明，`wsl --update` 用于更新 WSL，`wsl --version` 用于查看 WSL 及其组件版本。(Microsoft Learn)

c) 查看可安装的 Ubuntu 版本

执行：

```
wsl --list --online
```

也可以写成：

```
wsl -l -o
```

你要找的是：

```
Ubuntu-22.04    Ubuntu 22.04 LTS
```

Ubuntu 官方 WSL 文档示例中，可安装列表包含 `Ubuntu-22.04 Ubuntu 22.04 LTS`，并说明可以用列表里的 `NAME` 安装指定版本。(Ubuntu Documentation)

用管理员 PowerShell 执行：

```
ipconfig /flushdns
netsh winsock reset
netsh winhttp reset proxy
```

执行完后重启电脑，再试：

```
wsl --list --online
```

---

d) 安装 Ubuntu 22.04 LTS

执行：

```
wsl --install -d Ubuntu-22.04
```

如果你的 Windows / WSL 使用的是新版语法，也可能支持：

```
wsl --install Ubuntu-22.04
```

我建议用第一种，兼容性更好：

```
wsl --install -d Ubuntu-22.04
```

Microsoft 官方文档说明，`wsl --install -d <Distro>` 可以指定要安装的 Linux 发行版；如果没指定，默认会安装 Ubuntu。(Microsoft Learn)

如果下载卡在 0% 或 Microsoft Store 下载异常，试试：

```
wsl --install --web-download -d Ubuntu-22.04
```

Microsoft 文档也给出过这个处理方式：如果安装卡在 0.0%，可以加 `--web-download -d <DistroName>`。(Microsoft Learn)

---

e) 重启 Windows

安装完成后，按提示重启电脑。

重启后，再打开 PowerShell，执行：

```
wsl -l -v
```

出现类似：

```
  NAME            STATE           VERSION
* Ubuntu-22.04    Stopped         2
```

Microsoft 文档说明，可以用 `wsl --list --verbose` / `wsl -l -v` 查看已安装发行版及其 WSL 版本。(Microsoft Learn)

---

f) 如果 Ubuntu-22.04 不是 VERSION 2，改成 WSL2

如果你看到：

```
Ubuntu-22.04    Stopped    1
```

执行：

```
wsl --set-version Ubuntu-22.04 2
```

并把以后新装的发行版默认设为 WSL2：

```
wsl --set-default-version 2
```

Microsoft 文档说明，`wsl --set-version <distribution name> <versionNumber>` 可以把某个发行版设置为 WSL 1 或 WSL 2，`wsl --set-default-version 2` 可以把新发行版默认设为 WSL2。(Microsoft Learn)

g) 设置默认 Ubuntu 发行版

如果你只装了一个 Ubuntu，可以跳过。

如果你电脑里有多个 WSL 发行版，建议设定默认版本：

```
wsl --set-default Ubuntu-22.04
```

以后直接输入：

```
wsl
```

就会进入 Ubuntu 22.04。

Microsoft 文档说明，`wsl --set-default <Distribution Name>` 可以设置默认 WSL 发行版。(Microsoft Learn)

2.2.2.Ubuntu系统要求

a) 第一次进入 Ubuntu 22.04

执行：

```
wsl -d Ubuntu-22.04
```

或者从开始菜单搜索：

```
Ubuntu 22.04
```

第一次启动时，Ubuntu 会让你创建 Linux 用户名和密码。Microsoft 文档说明，首次启动新安装的 Linux 发行版时，会解压文件并要求创建用户账户和密码。(Microsoft Learn)

例如：

```
Enter new UNIX username: yxs
New password:
Retype new password:
```

注意：输入密码时屏幕不会显示字符，这是正常的。

---

b) 用 root 进入 Ubuntu

在 **PowerShell** 执行：

```
wsl -d Ubuntu-22.04 -u root
```

进入后执行：

```
whoami
```

应该输出：

```
root
```

Microsoft 文档说明，可以用 `wsl --distribution <Distribution Name> --user <User Name>` 指定发行版和用户进入，例如指定 `root`；如果用户不存在才会报错，而 `root` 是 Linux 里默认存在的用户。

---

c) 把这个 WSL 设置成默认 root + systemd

在 root 里面执行：

```
cat > /etc/wsl.conf <<'EOF'
[boot]
systemd=true

[user]
default=root
EOF

cat /etc/wsl.conf
```

Microsoft 的 WSL 配置文档说明，`/etc/wsl.conf` 是每个发行版自己的配置文件，可以配置 systemd 和默认用户；`[user] default=...` 用来指定 WSL 会话启动时使用哪个用户 。Docker 专用入口也正是用 `-u root` 进入 `/root/workspace` 并启动 Docker 服务

---

d) 把这个 WSL 设置成默认 root + systemd

进入 Ubuntu 后做基础更新

进入 Ubuntu 终端后，执行：

```
sudo apt update
sudo apt upgrade -y
```

再装一些常用工具：

```
sudo apt install -y curl wget git vim nano unzip ca-certificates gnupg lsb-release
```

确认 Ubuntu 版本：

```
cat /etc/os-release
```

你应该能看到类似：

```
VERSION="22.04.x LTS (Jammy Jellyfish)"
```

为后续 Docker Engine 做准备：启用 systemd

DataChallenging 本地课程环境采用 **WSL Ubuntu 内的 Docker Engine** 运行方式，不依赖 Docker Desktop，所以建议现在就启用 `systemd`。目标是在 WSL 内安装 Docker Engine 与 Compose 插件。

先在 Ubuntu 里检查：

```
ps -p 1 -o comm
```

如果输出已经是：

```
systemd
```

这一步可以跳过。

如果不是，执行：

```
sudo tee /etc/wsl.conf > /dev/null <<'EOF'
[boot]
systemd=true
EOF
```

然后退出 Ubuntu：

```
exit
```

回到 **Windows PowerShell**，执行：

```
wsl --shutdown
wsl -d Ubuntu-22.04/wsl -d Ubuntu
```

再检查：

```
ps -p 1 -o comm=
```

Microsoft WSL 配置文档说明，启用 systemd 需要在 `/etc/wsl.conf` 中加入 `[boot]` `systemd=true`，然后从 PowerShell 执行 `wsl.exe --shutdown` 重启 WSL 实例。(Microsoft Learn)

检查 WSL 与 Ubuntu 安装结果

2.2.3 检查最终状态

在 PowerShell 执行：

```
wsl -l -v
wsl --status
```

期望：

```
Ubuntu-22.04    Running 或 Stopped    2
```

进入 Ubuntu：

```
wsl -d Ubuntu-22.04
```

在 Ubuntu 里执行：

```
whoami
pwd
cat /etc/os-release
ps -p 1 -o comm=
```

理想结果：

```
Ubuntu 22.04.x LTS
systemd
```

2.2.4 最小完整版总结

在 **PowerShell 管理员模式**执行：

```
wsl --update
wsl --list --online
wsl --install -d Ubuntu-22.04
```

重启 Windows 后：

```
wsl -l -v
wsl --set-version Ubuntu-22.04 2
wsl --set-default Ubuntu-22.04
wsl -d Ubuntu-22.04
```

进入 Ubuntu 后：

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y curl wget git vim nano unzip ca-certificates gnupg lsb-release

cat /etc/os-release
```

为后续 Docker Engine 启用 systemd：

```
sudo tee /etc/wsl.conf > /dev/null <<'EOF'
[boot]
systemd=true
EOF

exit
```

回到 PowerShell：

```
wsl --shutdown
wsl -d Ubuntu-22.04
```

再在 Ubuntu 里确认：

```
ps -p 1 -o comm=
```

看到：

```
systemd
```

就说明 **Windows + WSL2 + Ubuntu 22.04 LTS** 基础环境完成。下一步才是在 Ubuntu 里安装 Docker Engine。

2.3 Docker Engine 与 Docker Compose 安装

方案一：通过 Docker 官方软件源安装

Docker 官方 Ubuntu 安装文档也是先配置 Docker apt 仓库，再安装 `docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`。(Docker Documentation)

🐬 ———— Docker Engine & Docker Compose ———— 🐬

2.3.1 确认当前状态

在 Ubuntu 里执行：

```
whoami
ps -p 1 -o comm=
cat /etc/os-release
```

理想结果类似：

```
root
systemd
Ubuntu 22.04.x LTS
```

清理可能存在的旧 Docker 包

```
for pkg in docker.io docker-doc docker-compose docker-compose-v2 \
  podman-docker containerd runc; do
  apt-get remove -y $pkg
done
```

更新 apt，并安装基础依赖

```
apt-get update

apt-get install -y ca-certificates curl gnupg
```

2.3.2 走Docker官方源/Ubuntu仓库

a) 添加 Docker 官方 GPG key

```
install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc

chmod a+r /etc/apt/keyrings/docker.asc
```

b) 添加 Docker 官方 apt 源

```
cat > /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

apt-get update
```

这一步对应你文档里的 Docker 官方仓库安装流程，核心就是写入 `/etc/apt/sources.list.d/docker.sources`，然后 `apt-get update`。

---

c) 安装 Docker Engine + Compose 插件

```
apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Docker 官方文档给出的最新版本安装包名也是这一组：`docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin`。(Docker Documentation)

---

d) 启动 Docker 服务并设置开机启动

```
systemctl enable --now docker

systemctl status docker --no-pager
```

看到类似：

```
active (running)
```

就说明 Docker daemon 已启动。Docker 官方文档也说明安装后应检查 `systemctl status docker`，如果没运行就用 `systemctl start docker` 启动。(Docker Documentation)

---

e) 验证 Docker 和 Compose

```
docker version
docker compose version
docker ps
docker run --rm hello-world
```

你文档里的验证项也是：`docker version` 同时看到 Client 和 Server，`docker compose version` 能输出版本号，`docker ps` 正常显示表头，`hello-world` 输出成功信息 。Docker 官方也推荐用 `docker run hello-world` 验证安装成功。(Docker Documentation)

---

f) 总结：可执行版脚本

```
whoami
ps -p 1 -o comm=
cat /etc/os-release

for pkg in docker.io docker-doc docker-compose docker-compose-v2 \
  podman-docker containerd runc; do
  apt-get remove -y $pkg
done

apt-get update
apt-get install -y ca-certificates curl gnupg

install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc

chmod a+r /etc/apt/keyrings/docker.asc

cat > /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

apt-get update

apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

systemctl enable --now docker
```

```
systemctl status docker --no-pager

docker version
docker compose version
docker ps
docker run --rm hello-world
```

方案二：通过 Ubuntu 仓库安装

你现在要走的是 **Ubuntu 仓库版 Docker**，不添加 Docker 官方源，也不写 `/etc/apt/sources.list.d/docker.sources`。

Ubuntu 官方包页面也显示 Ubuntu 22.04 Jammy 仓库里有 `docker.io` 包，属于 `universe` 组件。(Ubuntu 软件包搜索)

先执行这一整段：

```
# 1. 如果之前添加过 Docker 官方源，先删除，避免混用
rm -f /etc/apt/sources.list.d/docker.list
rm -f /etc/apt/sources.list.d/docker.sources
rm -f /etc/apt/keyrings/docker.asc
rm -f /etc/apt/keyrings/docker.gpg

# 2. 清理 apt 缓存并更新
apt-get clean
apt-get update

# 3. 确保 universe 仓库可用
apt-get install -y software-properties-common
add-apt-repository -y universe

# 4. 再更新一次
apt-get update

# 5. 查看 Ubuntu 仓库里 Docker 包是否可用
apt-cache policy docker.io docker-compose-v2 docker-compose

# 6. 使用 Ubuntu 仓库安装 Docker 和 Compose v2
apt-get install -y docker.io docker-compose-v2

# 7. 启动 Docker 服务
systemctl enable --now docker

# 8. 验证
systemctl status docker --no-pager
docker version
docker compose version
```

```
docker ps
```

Docker 官方文档也提醒，`docker.io`、`docker-compose`、`docker-compose-v2` 这类包是 Linux 发行版提供的包，和 Docker 官方源里的 `docker-ce` 等包不要混装；所以你现在选择 Ubuntu 仓库路线，就不要再同时保留 Docker 官方 apt 源。(docs.docker.com)

2.4 启动 DataChallenging 本地环境

进入课程部署目录

2.4.1 启动 Docker

进入 WSL Ubuntu 后执行：

```
systemctl start docker.service
systemctl status docker --no-pager
```

如果看到：

```
active (running)
```

说明 Docker 服务正常。

2.4.2 首次加载镜像并启动课程容器

课程离线包、Docker Compose 目录和镜像文件名可能随课程发布版本变化，因此**以教师实际发放的目录和文件名为准**。

例如：

```
cd /root/BigData-v10.1.XX
```

确认当前目录中存在 Compose 配置：

```
ls
```

应能看到：

```
docker-compose.yml
```

2.4.3 打开课程系统

如果课程环境还没有创建容器，执行：

```
docker load -i images/bigdata-student-10.1.19.image.tar
docker compose up -d
docker compose ps
```

<课程镜像文件>.tar的位置存放在当前工程项目BigData-v10.1.19下的**images**文件夹

如果课程提供的是离线 Docker 镜像，应先按照教师提供的文件名导入：

```
docker load -i <课程镜像文件>.tar
```

> 不要自行猜测镜像名或版本号。课程升级后，以实际发放包中的 Compose 文件和镜像文件为准。

2.4.4 日常快速启动

如果只是关机、重启 WSL 或容器停止：

```
cd /root/BigData-v10.1.19

docker compose up -d
docker compose ps
```

如果之前执行过：

```
docker compose down
```

容器已经被删除，则重新执行：

```
docker compose up -d
docker compose ps
```

2.4.5 一键启动

为了方便加载镜像，本系统提供直接一键启动Docker

```
scripts/install.sh
```

执行结果

```
[install] loading bigdata-student:10.1.XX image ...
Loaded image: bigdata-student:10.1.XX
[install] starting container ...
[+] Running 1/1
 ✓ Container bigdata-student  Running

                                        0.0s
[install] doctor:
[+] Running 1/1
 ✓ Container bigdata-student  Running0.0s
[doctor] PASS  python_version  3.12.15
[doctor] PASS  runtime_policy  policy/runtime_policy.json 合法
[doctor] overall=PASS
[install] done. 用 bin/datac --help 查看课程 CLI。
```

检查 DataChallenging CLI

进入课程的工作区目录：

```
cd /root/workspace/
```

执行：

```
bin/datac --help
```

成功标志：

- 命令可以正常执行；
- 可以看到课程 CLI 的帮助信息；
- 没有出现 `command not found`；
- 后续可以执行 `run` 和 `build`。

如果提示：

```
codesetarena: command not found
```

说明当前 shell 中没有找到课程 CLI。此时应优先检查：

1. 是否进入了正确的课程环境；
2. 是否需要进入课程容器执行；
3. 是否需要激活课程提供的 Python/Conda 环境；
4. 教师发布包中是否提供了 CLI 安装脚本。

不要自行修改课程接口名称。

2.5 认识课程项目

明确工作区目录

`workspace` 作为两个阶段工作区 + 两个原始恢复包。

课程镜像已经提前准备好两个阶段的工作目录，不再需要手动创建 workspace，也不需要第一次使用时手动解压模板。

应在课程容器的 /workspace 工作目录下执行操作。**注意：** 后续 `bin/datac` 命令均在容器内的 `/workspace` 目录执行，不是在 WSL 宿主机直接执行。

1. 安装与配置环境

```
bash scripts/install.sh
```

预期状态:

```
[install] loading bigdata-student:10.1.19 image ...
Loaded image: bigdata-student:10.1.19
[install] starting container ...
[+] Running 2/2
 ✓ Network bigdata-student_default  Created
                                 1.7s
 ✓ Container bigdata-student        Started
                                14.4s
[install] doctor:
[+] Running 1/1
 ✓ Container bigdata-student  Running0.0s
[doctor] PASS  python_version  3.12.15
[doctor] PASS  runtime_policy  policy/runtime_policy.json 合法
[doctor] overall=PASS
[install] done. 用 bin/datac --help 查看课程 CLI。
```

手动创建.env(最终.env是在workspace目录下)

```
cp workspace/.env.example workspace/.env
```

.env 和 .env.example 要求是存在于 workspace/ 根目录。

情况 A： .env 还不存在（首次配置）
 nano workspace/.env # 或 vim，填入三项

然后填模型配置项的三个值：

 LLM_BASE_URL=https://<你的网关>
 LLM_API_KEY=sk-...
 LLM_MODEL=<模型名>

情况 B： .env 已存在，想测试不同模型 —— 不要 cp 直接编辑补充新模型的配置项即可： nano workspace/.env

2. 验证 Reference 标准答案

```
bin/datac reference run --phase P1
```

预期状态：`PASS`

3. 运行 Agent v1

```
bin/datac run --workspace /workspace --phase P1
```

运行证据保存在：

```
/workspace/runs/phase1/<Run_ID>/result.json
```

4. 构建 Phase 1 正式提交包

```
bin/datac build --phase P1 --student-id <学号>
```

正式总包：

```
/workspace/store/packages/bigdata_p1_<学号>.tar.gz
```

总包包含：

```
Tasks_<学号>.tar.gz
Agent_<学号>.tar.gz
metadata.json
```

5. 验证正式提交包

```
bin/datac submission validate \
  --input /workspace/store/packages/bigdata_p1_<学号>.tar.gz \
  --phase P1
```

6. 导入 Exposure 包

```
bin/datac exposure import \
  --package <BigData_Exposure_Rxxxxxx.tar.gz> \
  --workspace /workspace
```

将占位文件名替换为实际助教包路径。

7. 检查任务分配

```
bin/datac exposure status --workspace /workspace
```

需要确认已分配 10 道任务。

8. 运行 Exposure（全量）

```
bin/datac exposure run --workspace /workspace
```

Exposure 固定使用：

```
phase1/agent.py
```

不会自动切换到 `phase2/agent.py`。

9. 运行 Exposure（单题）

```
bin/datac exposure run \
  --workspace /workspace \
  --task AT005 \
  --force
```

`AT005` 仅为示例，实际应使用已分配的任务编号。

10. 验证 Task v2 标准答案

```
bin/datac reference run --phase P2
```

目前测试结果：`PASS`，`answer=30`。

11. 运行 Agent v2

```
bin/datac run --workspace /workspace --phase P2
```

12. 构建 Phase 2 正式包

```
bin/datac build --phase P2 --student-id <学号>
```

13. 验证 Phase 2 正式包

```
bin/datac submission validate \
  --input /workspace/store/packages/bigdata_p2_<学号>.tar.gz \
  --phase P2
```

工作目录

```
workspace/
|
├── .env
├── .env.example                #实际API配置（首次复制生成）cp workspace/.env.example
workspace/.env
├── README.md
|
├── phase1/                     # 阶段一默认工作区，镜像中已提前解压
|   ├── task.json
|   ├── data.csv
|   ├── reference.py
|   ├── agent.py
|   └── metadata.json           # 强调必须填写自己的学号；"student_id": 学号
```

```
|
├── phase2/                        # 阶段二默认工作区，镜像中已提前解压
|   ├── task.json
|   ├── data.csv
|   ├── reference.py
|   ├── agent.py
|   ├── metadata.json              # 强调必须填写自己的学号："student_id": 学号
|   ├── review-comments.json
|   └── 实验报告模板.docx
|
├── exposure/
|   ├── assignment.json
|   |
|   ├── AT028/
|   |   ├── task.json
|   |   ├── data.csv
|   |   └── reference.py
|   |
|   ├── AT035/
|   |   ├── task.json
|   |   ├── data.csv
|   |   └── reference.py
|   |
|   └── ...
|
├── runs/
|   ├── phase1/
|   ├── phase2/
|   └── exposure/
|
└── store/
    ├── packages/
    └── builds/
```

内容	作用
`phase1/`	阶段一工作目录，已经提前解压好 Phase 1 默认模板
`phase2/`	阶段二工作目录，已经提前解压好 Phase 2 默认模板
`BigData-Phase1.tar.gz`	Phase 1 原始模板备份，用于恢复阶段一工作区
`BigData-Phase2.tar.gz`	Phase 2 原始模板备份，用于恢复阶段二工作区

课程已经为两个阶段分别准备独立工作区：

```
phase1/     第一次正式提交使用
phase2/     最终提交使用
```

两个目录相互独立。

2.5.1 阶段一：Task-Agent 初始方案

阶段一直接进入：

```
cd workspace/phase1
```

`phase1/` 中已经提供完整的默认 Task + Agent 工作模板，

2.5.2 阶段二：Task-Agent 最终方案

阶段二直接进入：

```
cd workspace/phase2
```

`phase2/` 同样由镜像提前提供完整的 Phase 2 默认模板。

镜像同时保留两个阶段的原始 ZIP 模板，就是为了让方便随时能够恢复。

> `phase1_template.tar.gz` 和 `phase2_template.tar.gz` 是课程提供的原始模板备份。
> 如果在实验过程中误删文件、修改错误或希望重新开始，可以使用对应 ZIP 恢复工作目录。：

不同阶段只操作自己的目录，命令必须显式指定 `--phase`。

建议每个阶段都在自己的工作区内开发、测试和提交，两个阶段互不覆盖。

测试标准答案：

```
bin/datac reference run --phase phase2
```

运行自己的 Agent：

```
bin/datac run --workspace /workspace --phase P2
```

明确失败模式

FM（Failure Mode）用于说明设计的 Task 主要希望考查 Agent 的哪一种错误类型。

例如，一道题要求先筛选数据再计算平均值，如果希望重点考查 Agent 是否能够正确理解聚合计算规则，就可以选择 `FM_AGGREGATION`。

需要注意：**FM 是在出题时候预先设定的目标失败模式，不代表 Agent 实际运行时一定会出现这种错误。**

必须根据自己设计的 Task，选择最匹配的一种失败模式，并按照系统规定的格式提交。失败模式必须填写。

1. FM 必须从系统规定的 14 种类型中选择

目前系统支持以下 14 种标准失败模式：

编号	失败模式	必须填写的 Failure Mode Key
FM01	需求理解偏差	`FM_REQUIREMENT`
FM02	Schema 理解错误	`FM_SCHEMA`
FM03	数据类型处理错误	`FM_DATA_TYPE`
FM04	筛选或排序错误	`FM_FILTER_SORT`
FM05	Join / 关联错误	`FM_JOIN`
FM06	聚合口径错误	`FM_AGGREGATION`
FM07	时间与时区错误	`FM_TIME_TIMEZONE`
FM08	缺失值处理错误	`FM_MISSING_VALUE`
FM09	异常值或边界处理错误	`FM_OUTLIER_BOUNDARY`
FM10	重复记录处理错误	`FM_DUPLICATE_RECORD_HANDLING`
FM11	规划与分解错误	`FM_PLANNING_DECOMPOSITION`
FM12	验证不足	`FM_VERIFICATION`
FM13	数值精度与舍入错误	`FM_NUMERICAL_PRECISION_ROUNDING`
FM14	提示注入或不可信数据	`FM_PROMPT_INJECTION`

特别注意：

- 每道 Task 必须选择一个主要失败模式。
- 必须填写第三列中的完整英文 Key。
- 第一列的 FM01、FM02 等编号仅用于展示和分类，不能作为提交值。
- 不允许自行创建新的失败模式。
- 不允许使用数字编号、简写或小写形式代替标准 Key。

例如：

正确：

```
FM_AGGREGATION
```

错误：

```
FM06_AGGREGATION
FM_06_AGGREGATION
fm_aggregation
FM_BOGUS
```

强调正式提交校验只接受标准 Key。

2. FM 应该填写在哪里?

FM 必须填写在 `metadata.json` 的 `hypothesized_failure_mode` 字段中。

例如:

```
{
  "hypothesized_failure_mode": "FM_AGGREGATION"
}
```

以上仅展示 FM 字段的正确写法，不代表完整的 `metadata.json` 只需要这一个字段。其他字段仍须按照课程提供的模板保留。

3. 应该如何选择合适的 FM?

建议先确定自己设计的 Task 主要考查 Agent 的什么能力，再选择对应的失败模式。

下面给出一些简单示例:

失败模式	示例任务或潜在错误
需求理解偏差	题目要求统计用户数量，Agent 却统计了记录数量
Schema 理解错误	将 `amount` 字段误认为数量，而不是金额
数据类型处理错误	把数值字符串当作普通文本处理
筛选或排序错误	题目要求先取最新记录，Agent 却先筛选状态
Join / 关联错误	按错误的关联字段连接数据
聚合口径错误	应按用户分组求和，却直接计算全表总和
时间与时区错误	未按题目要求统一时间或处理时区
缺失值处理错误	题目要求缺失值按 0 计算，Agent 却直接跳过
异常值或边界处理错误	将金额等于阈值的记录错误排除
重复记录处理错误	未按题目要求去除重复记录
规划与分解错误	多步骤计算时遗漏其中一个必要步骤
验证不足	得到中间结果后未检查是否符合最终统计口径

失败模式	示例任务或潜在错误
数值精度与舍入错误	应在最终结果保留两位小数，却提前对中间结果舍入
提示注入或不可信数据	将 CSV 中不可信的文本内容误当作需要执行的指令

以上是帮助理解不同 FM 的示例，不代表选择该标签后，Agent 就一定会出现对应错误。

4. 一道 Task 涉及多个 FM，应该怎么选?

一道数据分析题可能同时涉及筛选、排序、重复记录和聚合。

例如：

题目要求先选取每个用户的最新记录，再筛选状态为 ACTIVE 的用户，最后统计符合条件的用户数量。

这道题既涉及排序，也涉及筛选和聚合。

但是，如果最希望考查的是 Agent 是否能够正确执行“先选择最新记录，再判断状态”的操作顺序，那么应优先选择：

```
FM_FILTER_SORT
```

而不是因为最终使用了计数操作，就选择 `FM_AGGREGATION`。

建议按照以下原则选择：

- 首先明确题目最希望考查的错误类型。
- 再判断哪一种 FM 最能代表这道题的主要难点。
- 如果涉及多种处理操作，优先选择最关键、最具有区分性的错误类型。
- 不要为了选择某种 FM，故意设计不合理或存在歧义的题目。

原则：FM 应当反映 Task 的主要考查目标，而不是简单列举 Task 使用了哪些数据处理操作。

5. FM 必须与 Task 的实际设计相匹配

填写某个 FM 后，题目本身应当具有考查这种能力的合理依据。

例如，选择 `FM_MISSING_VALUE`，但 `data.csv` 中完全没有缺失值，题目也不涉及缺失值处理，那么这个失败模式就缺乏相应的设计依据。

同样，选择 `FM_TIME_TIMEZONE`，但题目既不涉及时间字段，也不涉及任何时间计算，那么这一分类通常也不合适。

需要特别区分：

可以主动设计容易让 Agent 出错的数据和处理规则，但不能故意把题目写错。

例如，允许设置：

- 先筛选和先排序会得到不同结果的数据。
- 包含缺失值和异常值的数据。
- 包含重复记录或临界值的数据。
- 需要多步骤推理才能得到正确结果的任务。

但必须保证：

- 题目要求明确。
- 数据能够支持正确计算。
- `reference.py` 按照题目要求实现。
- `answer` 是正确且可独立复核的标准答案。

6. FM 不会提前告诉被测 Agent

FM 属于出题者的设计信息，不是 Agent 解题所必需的公开信息。

在正式匿名评测中，系统会从 Task 的公开内容中移除 FM 字段，避免提前向被测 Agent 暴露题目的预设考查方向。

因此：

- 不要在 `task.json` 中额外填写 FM 字段。
- 不要通过嵌套 JSON 或其他变形字段隐藏 FM。
- 不要在公开题目中直接泄露标准答案或私有设计信息。
- 不要尝试通过额外文件向被测 Agent 传递 FM。

Task 的匿名版本应当只向 Agent 提供允许公开的任务信息和数据。

FM 则由教师端保留，用于后续实验统计与分析。

如果匿名 Task 中泄露了 FM 字段，系统可能报出 `EXPOSURE_LEAK`。

7. 预设 FM 不等于实际发生的 FM

填写的 `hypothesized_failure_mode` 只是对任务难点的预先判断。

例如，为一道题选择：

```
FM_FILTER_SORT
```

正式测试后，可能出现以下情况：

- Agent 正确完成任务，没有发生错误。
- Agent 正常返回错误答案，经过分析确认属于筛选顺序错误。
- Agent 正常返回错误答案，但实际原因是需求理解错误。
- Agent 因模型接口故障或资源超限而没有正常完成任务。

这些情况不能全部视为发生了 `FM_FILTER_SORT`。

系统会先区分 Agent 是否正常完成执行：

运行结果	含义	是否属于 FM 分析对象
`completed`	Agent 正常返回正确答案	不作为失败样本
`analysis_failure`	Agent 正常返回答案，但答案不正确	是，需要进一步分析实际错误

运行结果	含义	是否属于 FM 分析对象
`internal_error`	API、Token、资源或运行环境等原因导致执行异常	否，不计入 FM 统计

特别注意：

Agent 返回错误答案，并不能直接证明发生了预设的 FM。

需要进一步检查 Agent 的解题过程、计算结果和可验证的错误证据，才能判断实际失败类型。

助教端后续可以对比“任务包预设的 FM”和“Agent 实际发生的 FM”，分析不同 Task 的错误诱发效果。

8. Phase 1 与 Phase 2 的填写要求

Phase 1 和 Phase 2 的 `metadata.json` 均须包含合法的 `hypothesized_failure_mode`。

如果 Phase 2 修改了题目或数据，应重新检查当前 FM 是否仍然与任务设计一致。

不能因为 Phase 1 已经填写过 FM，就忽略 Phase 2 的对应检查。

此外，FM 只保留在外层的 `metadata.json` 中。

正式生成的 `Tasks_<学号>` 包内只包含：

```
Tasks_<学号>/
├── task.json
├── data.csv
└── reference.py
```

不包含 `metadata.json`，也不在内部 `task.json` 中额外加入 FM 字段。

9. 提交前必须自查

检查项目	要求
FM 是否填写	必须填写，不允许缺失
填写位置	仅在 `metadata.json` 中
字段名称	`hypothesized_failure_mode`
Key 格式	必须使用 14 种标准 Key 之一
数字编号	不允许使用 FM01、FM06 等编号形式作为提交值
自定义类别	不允许自行增加
主要失败模式	每道 Task 选择一个最匹配的类别
与题目一致	FM 应能反映 Task 的主要考查目标
公开内容	不得在 `task.json` 或匿名 Task 中泄露 FM 字段

检查项目	要求
Phase 1 / Phase 2	两阶段均须正确填写
实际失败判断	不能直接把预设 FM 当作已验证的失败原因

最后强调：FM 是设计 Task 时选择的目标错误类型，不是提前确定的实验结论。

需要围绕某一种具有代表性的失败模式设计题目，保证数据、计算规则、Reference 和标准答案正确，再通过实际 Agent 测试检验这道题能否诱发相应错误。

题目可以有针对性，但不能通过模糊规则、错误数据或错误标准答案来制造失败。

明确课程要求

⭐ ⭐ ⭐ 课程任务约束

2.5.3 规定Task，Agent

Task：一道完整的数据分析题 ⭐ ⭐ ⭐

Task 是一套完整、可执行、可验证的任务材料。

每道 Task 必须包含以下三个文件task.json（题面） data.csv（数据表）和 reference.py（参考代码）：

```
phase1/
└── 20250001/
    ├── task/
    |   ├── task.json
    |   ├── data.csv
    |   └── reference.py
    |
    ├── agent/
    |   └── agent.py
    |
    └── metadata.json
```

三个文件的作用分别是：

文件	作用	要求
task.json	说明题目要求及标准答案信息	题意完整、规则明确
data.csv	提供题目所需的原始数据	数据真实、格式正确、可读取,

文件	作用	要求
reference.py	实现题目的标准求解过程	能够执行，得到正确答案

其中，`reference.py` 是这道题的标准求解程序。执行它所得到的结果，必须与 `task.json` 中的 `answer` 一致。

简单来说：Task 是你设计的一道能够通过程序执行和验证的数据分析题，而不是一道只有文字描述的问答题，Task是包含了三个文件：task.json（题面） data.csv（数据表）和 reference.py（参考代码）。

a) task.json 应该如何编写？

基本格式如下：

```
{
  "description": "一句话概括你的任务。",
  "task_statement": "完整说明数据、计算要求和最终输出格式。",
  "answer": 30
}
```

四个字段分别有明确要求。

1. description：任务简介

用一句话说明这道题要完成什么，不需要展开具体计算过程。

2. task_statement：完整任务说明

建议严格按照以下四个部分组织：

- 数据说明：有哪些数据文件、字段名称、字段含义和数据类型。
- 筛选或处理规则：具体需要筛选什么、如何排序、如何处理缺失值、重复值和异常数据等。
- 计算目标：需要进行什么统计、聚合、比较或计算。
- 输出要求：明确最终返回什么结果，结果的数据类型、格式和精度是什么。

特别注意：不能只写大致思路，必须把影响最终答案的规则说清楚。

例如，如果要求选择每个用户最新的一条记录，就必须明确日期相同时如何处理；如果要求计算平均值，就必须明确缺失值是否参与计算。

3. answer：标准答案

`answer` 必须来自实际执行 `reference.py` 的结果，而不是人工估算或直接填写一个预期数字。

提交前必须确认：

- reference.py 能够正常运行；
- reference.py 实际读取 data.csv；
- 输出结果符合 task_statement 的要求；
- 执行结果与 answer 完全一致。

b) data.csv 应该如何编写？

`data.csv` 是 Task 的数据文件，必须包含完成 `task_statement` 所需的数据。

自行设计数据内容，包括字段、记录数量、数值范围和异常情况，但必须满足系统规定的文件限制，并确保数据能够支持题目要求的计算。

1. 文件格式与大小限制

系统对 `data.csv` 有以下硬性要求：

检查项目	具体要求
文件名	必须命名为 `data.csv`
文件大小	不超过 1,048,576 字节（1 MB）
数据行数	严格少于 10,000 行，最多 9,999 行
数据列数	不超过 100 列
文件编码	必须使用 UTF-8 编码
文件格式	必须能够通过 Python 的 `pd.read_csv()` 正常读取

特别注意：

- **行数不包含表头。** 例如，第一行是字段名，后面有 9,999 行数据，符合行数限制。
- **文件大小按实际字节计算。** 即使行数和列数没有超限，文件超过 1,048,576 字节也会校验失败。
- **必须保存为 UTF-8 编码。** 使用 GBK 等其他编码可能导致文件读取失败。
- 以上限制是系统固定规则，不能通过修改 `.env` 配置绕过。

2. 数据字段与题目保持一致

`data.csv` 中的字段名称、字段含义和数据内容必须与 `task_statement` 一致。

例如，题目要求根据用户编号、日期和状态统计最新状态为 ACTIVE 的用户数量，那么数据表就应提供完成这一计算所需的字段。

示例：

```
user_id,date,status
U01,2026-01-01,ACTIVE
U01,2026-01-03,INACTIVE
U02,2026-01-02,ACTIVE
U03,2026-01-04,ACTIVE
```

对应的题目可以要求：

对每个用户，先按照日期选择最新的一条记录，再统计最新状态为 ACTIVE 的用户数量，最终输出一个整数。

该示例的正确答案为 `2`。

数据设计时应注意：

- 字段名称应清楚，建议使用明确的表头，避免重复字段名。
- 字段含义和数据类型应与题干描述一致。
- 题目需要用到的字段和数据必须实际存在。
- 不得要求 Agent 读取未提供的其他数据文件。
- 不得依赖作者本地文件、私人数据库或外部网络数据才能完成计算。

3. 缺失值、重复值与异常数据必须有明确规则

允许在 `data.csv` 中设置缺失值、重复记录、非法数值、日期异常和其他具有挑战性的情况。

这些数据可以用来考查 Agent 的数据处理能力，但不能让题目变得含糊不清。

例如：

```
user_id,amount
U01,100
U02,
U03,abc
U04,200
```

其中，`U02` 的金额缺失，`U03` 的金额无法正常转换为数字。

如果题目要求计算金额总和，就必须在 `task_statement` 中说明：

- 缺失金额如何处理；
- 非法数值如何处理；
- 是否需要排除某些记录；
- 最终使用什么数值精度。

同样，如果数据包含重复记录、相同日期或排序并列等情况，也必须说明对应的处理方式。

原则：允许设置容易出错的数据，但必须给出明确的处理规则。

不是要求每道题都包含所有异常情况，而是只要某种情况可能影响答案，就必须说明如何处理。

4. 数据必须能够支持正确、可复现的计算

`data.csv` 不能只是形式上满足文件要求，还必须与 `task.json` 和 `reference.py` 相互一致。

具体要求：

- `task_statement` 中规定的计算能够使用提交的数据完成。
- `reference.py` 必须实际读取 `data.csv`，按照题目要求计算。
- `task.json` 中的 `answer` 必须与正确执行参考程序的结果一致。
- 相同数据、题目和运行环境下，应该能够重复得到相同答案。
- 不得故意设计与题干矛盾的数据，或者通过错误数据制造不合理的标准答案。

特别注意：一道Task除了检查程序运行结果，还需要检查计算逻辑是否正确，必要时使用边界数据进行验证。

5. Phase 1 与 Phase 2 的数据要求

Phase 1 和 Phase 2 均须遵守上述文件限制。

如果 Phase 2 要求修改数据，应当直接修改对应的 `phase2/data.csv`，并重新执行 `reference.py`，核对 `task.json` 中的 `answer` 是否仍然正确。

不能只修改 `answer`，而不重新验证数据和参考程序之间的一致性。

6. 提交前必须自查

检查项目	要求
文件名称	`data.csv`，不得自行更名
文件大小	⩽ 1,048,576 B
数据行数	⩽ 9,999 行（不含表头）
数据列数	⩽ 100 列
编码格式	UTF-8
文件读取	`pd.read_csv()` 能够正常读取
字段一致性	字段名称和含义与题干一致
数据完整性	所需计算不依赖其他未提供数据
特殊情况	影响答案的异常和边界情况有明确处理规则
结果正确性	数据、Reference 和标准答案相互一致

最后强调：数据不能模糊；可以故意设置容易让 Agent 出错的情况，但不能故意制造错误标准答案。

系统不仅检查 CSV 是否符合文件格式和大小要求，后续还会检查数据是否真正支持题目计算，以及参考程序和标准答案是否正确。

c) reference.py 应该如何编写？

`reference.py` 是 Task 的参考求解程序，用于根据 `data.csv` 中的数据，按照 `task_statement` 规定的规则计算标准答案。

可以自行设计计算逻辑，但必须遵守系统规定的代码格式、运行限制和答案一致性要求。

1. 文件格式与代码限制

系统对 `reference.py` 有以下硬性要求：

检查项目	具体要求
文件名	必须命名为 `reference.py`
代码行数	严格少于 200 行，最多 199 行
Python 语法	必须能够通过 `ast.parse()` 语法检查

检查项目	具体要求
import 白名单	只允许导入 `pandas` 和 `decimal`
相对导入	禁止使用
文件读取	禁止使用 `open()` 和 pandas 的 `read_*()` 方法读取外部数据
文件写出	禁止使用 `to_csv()`、`to_excel()` 等方法写出数据
危险函数	禁止调用 `exec()`、`eval()`、`compile()`、`__import__()`
单次计算超时	`reference(df)` 必须在 1 秒内完成执行

特别注意：

- 不能导入 `numpy`、`os`、`json` 等白名单之外的模块。
- 可以使用 Python 允许的内置计算函数以及 pandas 提供的数据处理功能。
- 不允许读取其他 CSV、Excel、数据库或外部文件。
- 不允许通过网络请求、外部 API 或其他程序获取答案。
- 程序必须能够在课程统一运行环境中正常执行。

这些限制由系统自动检查，不是建议性的代码风格要求。

2. 必须定义 reference(df) 函数

系统要求 `reference.py` 中必须存在一个名为 `reference` 的可调用函数，而且参数名称必须是 `df`。

基本格式如下：

```python
def reference(df):
    # 按照题目要求处理数据
    result = int(df["amount"].sum())
    return result
```

其中：

- `df` 是系统从 `data.csv` 读取后传入的 pandas DataFrame。
- 函数内部按照题目要求进行筛选、排序、计算或聚合。
- 使用 `return` 返回最终计算结果。
- 返回值必须与 `task.json` 中的 `answer` 一致。

特别注意：

系统已经负责读取 `data.csv`，不需要在 `reference.py` 中再次读取文件。

因此，下面的写法不符合要求：

```python
import pandas as pd

def reference(df):
    data = pd.read_csv("data.csv")
    return int(data["amount"].sum())
```

正确做法是直接使用系统传入的 `df`：

```python
def reference(df):
    return int(df["amount"].sum())
```

函数名称和参数名称不能自行修改，也不能增加其他参数。例如，`solve(df)`、`reference(data)`、`reference(df, config)` 均不符合接口要求。

最终答案必须通过 `return` 返回，不能只使用 `print()` 输出。

3. reference.py 必须真正实现题目要求

参考程序不能只保证运行成功，还必须正确实现 `task_statement` 中规定的计算逻辑。

例如，题目要求：

对每个用户，先根据日期选择最新记录，再统计最新状态为 ACTIVE 的用户数量。

那么 `reference.py` 必须按照这个顺序处理数据。

不能先筛选 ACTIVE 状态，再选择最新记录，因为这两种计算方式可能得到不同结果。

编写时应注意：

- 筛选条件必须与题干一致。
- 排序、去重和分组顺序必须符合题目要求。
- 缺失值、异常值、重复记录和边界情况必须按照题干规定处理。
- 计算公式、数值精度及四舍五入规则必须与题干一致。
- 不得为了匹配预期答案而擅自改变计算规则。

不能直接硬编码标准答案。

例如：

```python
def reference(df):
    return 30
```

即使 `task.json` 中的 `answer` 也是 `30`，这种写法仍然不符合参考程序必须实际计算数据的要求。

正确做法是根据 `df` 中的数据完成计算。

4. 运行时间与执行要求

系统在执行 `reference(df)` 时设置了 **1 秒超时限制**。

因此，参考程序应尽量使用简单、明确、高效的数据处理方式。

建议：

- 优先使用 pandas 的筛选、排序、分组和聚合功能。
- 避免没有必要的多层循环和重复计算。
- 不要进行与题目无关的大规模计算。
- 不要使用无限循环、长时间等待或其他可能导致超时的操作。
- 不要依赖运行环境之外的文件、模块或服务。

如果函数执行超时或运行过程中出现异常，系统可能报出 `REFERENCE_ERROR`，导致提交失败。

此外，系统向函数传入的是数据副本，可以在函数内部进行必要的数据处理，但不能通过修改原始文件影响其他任务或评测结果。

5. 计算结果必须与 answer 一致

系统会真实执行 `reference.py`，再将计算结果与 `task.json` 中的 `answer` 比较。

例如：

`task.json`：

```
{
  "description": "计算所有记录的金额总和。",
  "task_statement": "给定 data.csv，其中 amount 表示金额。计算全部记录的 amount 总和，最终返回一个整数。",
  "answer": 30
}
```

`reference.py`：

```
def reference(df):
    result = int(df["amount"].sum())
    return result
```

假设 `data.csv` 中的金额分别为 `10` 和 `20`，则参考程序返回 `30`，与 `answer` 一致。

需要注意：

- 标准答案必须来自真实计算，而不能人工猜测或估算。
- 如果修改了数据或参考程序，必须重新运行并检查 `answer`。
- 系统对普通数值类型允许一定的数值归一化比较，例如整数 `30`、浮点数 `30.0` 和字符串 `"30"` 可能被判断为数值相等。
- 布尔值不进行数值归一化；列表、字典等结构化答案需要满足更严格的一致性要求。

要求仍然按照题干规定返回正确的数据类型，不要依赖系统的数值归一化机制来弥补格式问题。

如果返回结果与 `answer` 不一致，系统会报出 `ANSWER_MISMATCH`，导致打包失败。

6. 参考程序还需要通过独立正确性检查

需要特别强调：

reference.py 的计算结果与 answer 一致，不代表参考程序一定正确。

例如，题目要求先选择最新记录，再判断状态，但实际编写的参考程序采用了相反的操作顺序。

即使当前数据恰好使两种计算方式得到相同答案，程序仍然没有正确实现题意。

因此，课程后续还会检查：

- 参考程序是否完整实现题干要求。
- 是否存在筛选、排序、分组、聚合或其他处理逻辑错误。
- 是否遗漏影响计算结果的边界情况。
- 是否能够根据题干和数据独立复核标准答案。

Task Quality Gate 会尝试根据题面和数据独立求解，并与Task提供的 Reference 结果对账。

如果两者存在差异，需要进一步检查题目和参考程序，而不能简单地修改 `answer` 以通过校验。

7. 提交前必须自查

检查项目	要求
文件名	必须为 `reference.py`
代码行数	最多 199 行
语法检查	Python 语法正确
导入模块	仅使用 `pandas`、`decimal`
函数接口	必须定义可调用的 `reference(df)`
数据输入	直接使用系统传入的 `df`
结果输出	使用 `return` 返回计算结果
文件操作	不读取或写出外部文件
危险调用	不使用被禁止的动态执行函数
运行时间	函数调用在 1 秒内完成
计算逻辑	完整符合 `task_statement`
标准答案	返回结果与 `answer` 一致
可复现性	相同数据和环境下得到相同结果

最后强调：reference.py 是一份真正用于求解题目的参考程序，而不是保存标准答案的文件。

必须根据题干编写正确的计算逻辑，直接处理系统传入的 DataFrame，并返回实际计算结果。

系统会同时检查代码是否合法、程序能否运行、结果是否与 `answer` 一致，以及参考程序是否真正符合题目要求。

⭐ ⭐ ⭐ Agent：一道解决 Task 的程序

Agent 是专门针对给定 Task 编写的求解智能体程序，文件为：

```
agent/
└── agent.py
```

Agent 需要根据题目要求，读取提供的 CSV 数据，完成相应的数据处理和计算，并按照题目规定返回结果。

agent.py 应该如何编写？

`agent.py` 是必须提交的智能体程序，最后会用于读取其他人设计的 Task，并根据题目要求和提供的 CSV 数据，独立计算最终答案。

强调自行设计 Agent 的解题策略，例如直接调用大语言模型、使用 pandas 处理数据，或者让模型分步骤分析并执行计算。

但必须遵守系统规定的函数接口、资源预算和运行要求。设计agent.py的目标是正确解决 Task，

1. 文件格式与接口要求

系统对 `agent.py` 有以下硬性要求：

检查项目	具体要求
文件名	必须命名为 `agent.py`
Python 导入	程序必须能够正常导入
函数名称	必须定义可调用的 `run` 函数
函数参数	必须恰好为 `run(task, df, llm)`
参数顺序	`task`、`df`、`llm`，顺序固定
参数数量	只能有这三个参数，不能增加或减少
最终输出	必须通过 `return` 返回最终答案
文件数量	打包后的 `Agent_<学号>/` 中只能包含 `agent.py`

特别注意：

- 不能把 `run` 改成 `solve`、`main` 等其他名称。
- 不能将 `df` 改成 `data`，或者将 `llm` 改成 `model`。
- 不能自行添加其他参数或默认值参数。

- 不允许只通过 `print()` 输出最终答案。
- Agent 必须能够在课程统一 Runner 中正常执行。

系统会按照固定接口调用Python代码程序，因此必须保持函数名称和参数完全一致。

2. 必须定义 run(task, df, llm) 函数

Agent 的基本接口如下：

```python
def run(task, df, llm):
    # 根据 task 理解题目要求
    # 使用 df 中的数据进行分析
    # 必要时通过 llm 辅助推理

    result = ...  # 最终计算结果
    return result
```

上述代码只用于说明函数结构，不是可以直接运行的完整 Agent。

三个参数的含义分别是：

参数	含义
`task`	系统提供的公开任务信息，用于理解题目要求
`df`	系统读取 `data.csv` 后得到的 pandas DataFrame
`llm`	系统提供的大语言模型调用接口

其中，`df` 已经由系统读取并传入，因此 Agent 不需要自行打开 `data.csv`。

`llm` 由课程统一提供的接口函数，不需要自行配置模型 API Key 或接入额外模型。

3. Agent 应当能够解答不同类型的 Task

与 `reference.py` 不同，Agent 不是只用于解答自己提交的一道题。

正式评测时，Agent 还需要面对其他人设计的未知任务。

例如：

- 有的 Task 要求筛选数据后计算总和。
- 有的 Task 要求排序、去重后统计数量。
- 有的 Task 要求处理缺失值或异常数据。
- 有的 Task 涉及时间、重复记录或多条件判断。
- 有的 Task 要求按照指定精度返回最终结果。

因此，Agent 应当能够根据不同题目的要求，动态决定解题步骤。

不能把某一道题的标准答案直接写进代码，也不能只根据固定字段名称返回预设结果。

允许设计不同的智能体策略，但必须根据实际收到的题目和数据求解。

4. LLM 应当如何调用？

系统已经提供了统一的大语言模型接口。

Agent 应通过以下方法调用模型：

```
response = llm.complete(messages)
```

其中，`messages` 是消息列表，每条消息包含 `role` 和 `content`。

示例：

```
messages = [
    {
        "role": "system",
        "content": "你是一个数据分析助手。"
    },
    {
        "role": "user",
        "content": "请根据题目要求分析数据并计算答案。"
    }
]

response = llm.complete(messages)
```

这里仅展示模型调用方式，实际 Agent 需要把本次任务要求和必要的数据内容提供给模型，或者使用 pandas 工具完成计算。

消息角色支持：

- `system`：规定模型的行为和任务。
- `user`：提供题目、数据和计算要求。
- `assistant`：保存模型之前的回复，用于多轮交互。

可以根据自己的策略设计单轮或多轮模型调用，但只能使用平台正式提供的模型接口和资源额度。

5. Agent 的运行资源限制

系统对每次 Agent 执行设置了统一资源预算。

资源项目	系统限制
执行时间	90 秒，另有 10 秒启动宽限，总保护时间约 100 秒
内存	2048 MB
临时磁盘空间	512 MB
累计 Token	每次执行最多 10,000 Tokens
单次 LLM 请求超时	75 秒
LLM 超时重试	最多 1 次超时重试

这些限制由系统统一管理，不能自行修改资源，一经发现，将不算作成绩。

特别注意：

- **Token 是整次任务的累计预算**，不是每轮模型调用都可以使用 10,000 Tokens。
- 多轮调用、工具执行和数据处理均会占用有限的执行时间。
- 单次 LLM 请求即使允许超时重试，也仍然受到整个 Agent 执行时间限制。
- 不建议将完整的大型 CSV 直接发送给模型，应根据题目需要合理读取、筛选或汇总数据。
- 不要在程序中设置不必要的长时间等待或无限循环。
- 避免产生大量无关中间文件或占用过多内存。

如果超出时间、内存、磁盘或 Token 限制，系统将终止本次执行。

资源超限不会作为普通的工具观察结果返回给 Agent，让模型继续分析或重试。

因此，需要在设计 Agent 时合理安排调用次数和计算过程。

6. Agent 可以使用哪些工具?

可以使用系统正式提供的数据、模型接口和运行能力，但不能通过额外信息来源获取标准答案。

能力或行为	规范要求
读取公开任务要求	允许
处理系统传入的 DataFrame	允许
使用 pandas 进行计算	允许
调用平台提供的 `llm.complete()`	允许
多轮模型推理	允许，但受资源预算限制
使用本地数据筛选、排序、分组与聚合	允许
自行接入第三方模型 API	不允许
联网搜索题目答案	不允许
读取其他人的 Reference 或 Gold	不允许
读取未授权的本地文件或私人数据	不允许
修改原始任务数据以改变判分结果	不允许
直接硬编码已知标准答案	不允许

设计的智能体程序访问是在我们规定的运行范围之内，而不是任意访问文件、网络或额外服务。

设计智能体仍应遵守课程统一规定的数据访问范围、模型使用权限和评测要求。

7. 如何使用系统当中提供的 Agent 模板?

课程提供的标准 Agent 模板已经包含基于模型调用与 pandas 执行的基本解题流程。

模板要求模型每轮输出一种 JSON Action。

例如，执行 pandas 计算：

```
{
  "action": "execute_pandas",
  "code": "result = int(df['amount'].sum()); print(result)"
}
```

或者提交最终答案：

```
{
  "action": "final_answer",
  "answer": 30
}
```

在标准模板中：

- `execute_pandas` 用于执行数据处理代码。
- `final_answer` 表示模型已经得到最终结果。
- 执行结果会作为 Observation 返回，供模型继续判断。
- 如果生成的 Action 不是合法 JSON，模板会返回错误信息，供模型自行纠正。
- 模板对生成的 pandas 代码进行了基本安全检查，例如禁止直接导入模块以及使用 `open()`、`exec()`、`eval()`。
- 模板中的 pandas 执行环境主要提供 `df` 和 `pd` 两个对象。

特别注意：上述 JSON Action 是标准模板内部使用的交互格式，并不是平台强制要求所有 Agent 必须采用的实现方式。

可以在遵守 `run(task, df, llm)` 接口和系统资源限制的前提下，设计不同的智能体架构。

如果直接修改标准模板，建议保留已有的代码执行检查、异常处理和工具调用流程，避免因修改不当导致程序无法正常工作。

8. 最终答案应该如何返回?

Agent 必须根据 `task_statement` 要求返回最终结果。

例如：

题目输出要求	正确返回形式
返回整数	`30`
返回保留两位小数的结果	`"85.00"`
返回保留两位小数的结果	`"0.30"`
返回字符串	`"ACTIVE"`
返回布尔值	`True` 或 `False`，以题目及评测格式要求为准

特别注意：

- 如果题目要求整数，应返回整数或符合判分规则的整数文本。
- 如果要求固定小数位数，应保留对应的小数位。
- 不要在最终答案中额外添加解释、单位或 Markdown 代码块，除非题目明确要求。
- 不要把模型的分析过程当作最终答案。
- 不要只在控制台打印结果，必须通过 `run()` 的返回值提交。

例如，题目要求输出整数 `30`，不应返回：

```
"经过统计，最终答案为30。"
```

应直接返回：

```
30
```

系统会将 Agent 的返回结果与标准答案进行比较。

对于普通数值，判分器允许一定的数值归一化；但仍应按照题目规定的输出格式返回答案。

9. Agent 运行结果如何判定?

系统主要将运行结果分为以下三类：

结果类型	含义
`completed`	Agent 正常完成并返回正确答案
`analysis_failure`	Agent 正常返回答案，但与标准答案不一致
`internal_error`	模型 API、资源限制、程序崩溃或其他运行异常导致失败

这里需要特别区分：

Agent 算错题，与 Agent 程序运行失败，是两种不同情况。

例如，Agent 正常运行，但把一道排序题的结果从 `30` 算成 `29`，属于解题错误。

如果 Agent 超时、内存超限、Token 超限，或者模型接口调用失败，则属于执行异常。

课程会根据不同结果分析 Agent 的解题能力和失败原因。

10. Phase 1 与 Phase 2 的 Agent 使用规则

Phase 1 提交的 `agent.py` 是正式参与 Exposure 测试的 Agent 版本。

按照当前系统实现，即使 Phase 2 目录中存在新的 `agent.py`，Exposure 仍使用 `phase1/agent.py`。

因此：

- Phase 1 应提交能够正常运行的 Agent。
- 不要假设修改 Phase 2 的 `agent.py` 就会自动影响 Exposure 评测。

- Phase 1 和 Phase 2 的 `agent_id` 必须保持一致。
- `agent_id` 由 `metadata.json` 管理，格式为 `AGENT_<student_id>`。
- 正式打包时，`Agent_<学号>/` 目录中只能包含 `agent.py` 一个文件。

11. 提交前必须自查

检查项目	要求
文件名称	`agent.py`
文件导入	能够正常导入
函数接口	必须为 `run(task, df, llm)`
参数名称与顺序	严格保持一致
数据输入	使用系统提供的公开任务信息和 DataFrame
模型调用	使用 `llm.complete(messages)`
最终输出	通过 `return` 返回答案
时间预算	不超过平台规定的执行时间
内存预算	不超过 2048 MB
Token 预算	单次执行累计不超过 10,000
临时磁盘	不超过 512 MB
工具使用	不访问未经授权的外部服务或数据
解题能力	能够处理不同人设计的未知Tasks
打包格式	`Agent_<学号>/` 内只有 `agent.py`

最后强调：agent.py 是用来参加实际解题评测的智能体，不是某一道题的固定答案程序。

需要根据未知题目的要求，合理使用数据处理能力和平台提供的大语言模型，在规定资源预算内返回准确答案。

Agent 可以采用不同的实现策略，但必须遵守统一的接口、资源和数据访问规范，不能通过额外模型、外部搜索或读取标准答案获得不公平优势。

总览：Task和Agent核心要求

文件	核心要求
`task.json`	题目清楚、规则明确、答案可验证
`data.csv`	数据合法、完整，能支持题目计算
`reference.py`	正确实现题意，计算出真实标准答案

文件	核心要求
`agent.py`	面向未知题目，在统一资源预算内独立求解

⭐ ⭐ ⭐ 元信息管理文件填写 ⭐ ⭐ ⭐

`metadata.json` 是 Task 和 Agent 的元信息管理文件，用于记录学号信息、任务编号、智能体编号以及希望考查的失败模式。

与 `task.json`、`data.csv`、`reference.py` 和 `agent.py` 不同，`metadata.json` 不负责描述题目内容或实现计算，而是用于标识提交对象，确保 Phase 1 和 Phase 2 的任务与智能体能够正确对应。

必须严格按照系统规定的字段名称和取值填写，不能自行增加、删除或修改字段。

1. 文件位置与基本要求

每个 Phase 目录中都必须包含一份 `metadata.json`。

```
workspace/
├── phase1/
|   ├── task.json
|   ├── data.csv
|   ├── reference.py
|   ├── agent.py
|   └── metadata.json
|
└── phase2/
    ├── task.json
    ├── data.csv
    ├── reference.py
    ├── agent.py
    └── metadata.json
```

特别注意：

- 文件名必须为 `metadata.json`。
- 文件内容必须符合标准 JSON 格式。
- 字段名称区分大小写，必须与系统规定完全一致。
- Phase 1 和 Phase 2 的字段要求不同，不能直接复制后不作修改。
- `metadata.json` 是学号信息及 Task、Agent 编号的唯一管理文件，不应在其他文件中另行维护与之冲突的身份信息。

2. Phase 1 的 metadata.json 应该如何填写？

Phase 1 必须且只能包含以下五个字段：

```
{
  "workspace": "phase1",
  "student_id": "12561022",
  "task_id": "TASK_12561022",
  "agent_id": "AGENT_12561022",
  "hypothesized_failure_mode": "FM_AGGREGATION"
}
```

强调注明自己的学号，请在：

```
metadata.json
```

中填写自己的学号在以下三个字段：

```
  "student_id": "你的学号"
  "task_id": "TASK_你的学号",
  "agent_id": "AGENT_你的学号"
```

学号要填写在student_id，task_id和agent_id

五个字段分别有明确要求。

（1）workspace：所属阶段

用于说明当前文件属于哪个阶段。

Phase 1 必须填写：

```
"workspace": "phase1"
```

Phase 2 必须填写：

```
"workspace": "phase2"
```

不能填写其他名称，也不能将 Phase 1 的元信息误写成 `phase2` 。

（2）student_id：学号或唯一身份标识

填写自己的学号，例如：

```
"student_id": "12561022"
```

要求：

- 长度为 2～32 个字符。
- 只能包含英文字母、数字和下划线。
- 不允许空格、中文、连字符或其他特殊符号。
- 必须与构建命令中提供的 `--student-id` 一致。
- 必须与最终提交包对应的学号标识一致。

建议直接填写自己的真实学号，不要使用示例中的编号提交。

（3）task_id：任务编号

用于唯一标识所提交的 Task。

必须采用：

```
TASK_<student_id>
```

例如，学号为 `12561022`：

```
"task_id": "TASK_12561022"
```

不能自行修改前缀，也不能使用其他编号。

（4）agent_id：智能体编号

用于唯一标识所提交的 Agent。

必须采用：

```
AGENT_<student_id>
```

例如：

```
"agent_id": "AGENT_12561022"
```

`task_id` 和 `agent_id` 是两种不同的编号，不要混用。

（5）hypothesized_failure_mode：预期失败模式

表示自己设计的 Task 主要希望考查 Agent 在哪个环节容易出错。

必须从系统规定的 14 种失败模式中选择一项，填写完整的 Failure Mode Key。

目前允许的类型包括：

- `FM_REQUIREMENT`
- `FM_SCHEMA`
- `FM_DATA_TYPE`
- `FM_FILTER_SORT`
- `FM_JOIN`
- `FM_AGGREGATION`
- `FM_TIME_TIMEZONE`
- `FM_MISSING_VALUE`
- `FM_OUTLIER_BOUNDARY`
- `FM_DUPLICATE_RECORD_HANDLING`
- `FM_PLANNING_DECOMPOSITION`
- `FM_VERIFICATION`
- `FM_NUMERICAL_PRECISION_ROUNDING`
- `FM_PROMPT_INJECTION`

例如，题目主要考查分组统计、求和或平均值计算，可以选择：

```
"hypothesized_failure_mode": "FM_AGGREGATION"
```

特别注意：

- 必须填写系统规定的完整名称。
- 不能自行新增失败模式。
- 不使用带数字编号的名称，例如 `FM06_AGGREGATION`。
- 如果一道题涉及多个错误环节，应选择最主要、最希望考查的失败模式。
- 失败模式表示希望考查的能力弱点，不代表可以故意提供错误题干、错误 Reference 或错误标准答案。

3. Phase 2 的 metadata.json 应该如何填写?

Phase 2 在 Phase 1 的五个字段基础上，必须额外增加一个字段：

```
review-comments_id
```

完整示例如下：

```
{
  "workspace": "phase2",
  "student_id": "12561022",
  "task_id": "TASK_12561022",
  "agent_id": "AGENT_12561022",
  "hypothesized_failure_mode": "FM_AGGREGATION",
  "review-comments_id": "REVIEW_12561022"
}
```

其中：

- `workspace` 修改为 `phase2`。
- `student_id` 保持同一学号。
- `task_id` 必须与 Phase 1 完全相同。
- `agent_id` 必须与 Phase 1 完全相同。
- `hypothesized_failure_mode` 仍须使用合法的标准 Key。
- 新增的 `review-comments_id` 必须采用 `REVIEW_<student_id>` 格式。

特别注意：Phase 2 是同一个 Task 和 Agent 的第二阶段版本，不是重新创建新的 Task 或 Agent。

因此，不能将 Phase 2 的编号改成 `TASK_12561022_V2` 或 `AGENT_12561022_P2`。

系统会检查两个阶段的 `task_id` 和 `agent_id` 是否一致。

4. 不允许增加或缺少字段

`metadata.json` 使用严格字段校验。

Phase 1 只能有五个字段，Phase 2 只能有六个字段。

例如，下面的 Phase 1 写法不符合要求：

```
{
  "workspace": "phase1",
  "student_id": "12561022",
  "task_id": "TASK_12561022",
  "agent_id": "AGENT_12561022",
  "hypothesized_failure_mode": "FM_AGGREGATION",
  "name": "张三"
}
```

其中 `name` 不是系统允许的字段，即使其他内容全部正确，也会导致校验失败。

同样，不能遗漏 `student_id`、`task_id`、`agent_id` 或 `hypothesized_failure_mode`。

原则：系统规定几个字段，就填写几个字段，不要增加任何自定义信息。

5. 失败模式必须填写在 metadata.json 中

按照当前 v10.1.18 及之后的契约，`hypothesized_failure_mode` 已从 `task.json` 迁移到 `metadata.json`。

因此，当前 `task.json` 应采用以下形式：

```
{
  "description": "计算所有记录的金额总和。",
  "task_statement": "给定 data.csv，其中 amount 表示金额。计算全部记录的 amount 总和，最终返回一个整数。",
  "answer": 30
}
```

对应的失败模式写在 `metadata.json` 中：

```
{
  "workspace": "phase1",
  "student_id": "12561022",
  "task_id": "TASK_12561022",
  "agent_id": "AGENT_12561022",
  "hypothesized_failure_mode": "FM_AGGREGATION"
}
```

特别注意：

- `task.json` 中不能再出现 `hypothesized_failure_mode`。
- `metadata.json` 中必须存在 `hypothesized_failure_mode`。
- 必须使用不带数字编号的标准 Failure Mode Key。

- 不要混用新旧版本的 JSON 格式。

如果仍在 `task.json` 中填写该字段，系统会报出 `TASK_JSON_FORBIDDEN_HYPOTHESIZED_FAILURE_MODE`。

如果 `metadata.json` 中缺少该字段，则会报出 `METADATA_MISSING_HYPOTHESIZED_FAILURE_MODE`。

6. Phase 1 与 Phase 2 的身份信息必须保持一致

系统要求两个阶段属于同一个人操作(两个阶段学号务必一致)、同一 Task 和同一 Agent。

例如：

字段	Phase 1	Phase 2
`workspace`	`phase1`	`phase2`
`student_id`	`12561022`	`12561022`
`task_id`	`TASK_12561022`	`TASK_12561022`
`agent_id`	`AGENT_12561022`	`AGENT_12561022`
`hypothesized_failure_mode`	合法标准 Key	合法标准 Key
`review-comments_id`	不允许出现	`REVIEW_12561022`

系统在构建 Phase 2 提交包时，会重新读取 Phase 1 的元信息，并检查 `task_id` 和 `agent_id` 是否一致。

可以按课程要求修改 Phase 2 的任务内容，但不能重新生成 Task 或 Agent 身份编号。

7. 提交前必须自查

检查项目	要求
文件名称	必须为 `metadata.json`
JSON 格式	能够正常解析
字段数量	Phase 1 恰好 5 个，Phase 2 恰好 6 个
`workspace`	与所属阶段目录一致
`student_id`	2～32 位合法字符，与实际学号及构建参数一致
`task_id`	严格为 `TASK_<student_id>`

检查项目	要求
`agent_id`	严格为 `AGENT_<student_id>`
失败模式	必须使用 14 个标准 Key 之一
`review-comments_id`	仅 Phase 2 存在，格式为 `REVIEW_<student_id>`
P1/P2 连续性	两个阶段的 Task ID、Agent ID 必须相同
多余字段	不允许存在
缺失字段	不允许缺少
`task.json`	不能包含 `hypothesized_failure_mode`

最后强调：metadata.json 只负责管理提交身份与失败模式，不负责存放题目答案或实现计算逻辑。

必须按照规定填写完整且准确的字段，确保自己的 Task、Agent 和两个阶段的提交记录能够正确关联。

不要自行修改编号格式，也不要混用旧版与新版的失败模式字段规范。

数据挖掘课程小组作业

作业目的与总体要求

本次课程大作业考核采用小组项目作业形式，不再安排期末笔试。各小组需要围绕实际应用场景，自主选择数据挖掘案例，完成数据分析、算法应用、分布式计算方案设计以及算法可视化演示，综合考查对数据挖掘基本理论、算法原理和大数据处理技术的理解与应用能力。

本次作业不仅要求掌握数据挖掘算法的基本原理，更强调将算法应用于实际问题的能力，以及通过动画、可视化等形式展示算法运行过程的能力。

每个小组需要独立完成 **两个数据挖掘案例**。每个案例均应包含以下三个部分：

1. 数据分析案例描述；
2. 算法应用方案与算法流程描述；
3. 海量数据下大数据分布式计算解决方案。

两个案例依次组织，分别形成完整的分析、算法应用和分布式计算方案。

最终成果应体现数据挖掘算法从实际问题提出、数据处理、算法运行到结果分析，以及从单机计算向分布式计算扩展的完整过程。

第三部分：Phase 1 — Task 与 Agent 初始设计

3.1 本地测试与提交阶段1作业

a) 构建 Phase 1 提交包

在课程目录执行：

```
bin/datac build --phase P1 --workspace /workspace --student-id <学号>
```

系统会完成类似流程：

```
检查 Task 文件
      ↓
读取数据
      ↓
运行 Agent
      ↓
计算标准答案
      ↓
比较 Agent Output 与 Ground Truth
      ↓
生成运行结果
```

b) 测试 Task-Agent 初始设计

本地运行测试指令：

```
bin/datac run --workspace /workspace --phase P1
```

.env 应该如何配置?

`.env` 是 DataChallenging 项目的 LLM 运行配置文件，用于配置课程大语言模型网关的连接地址、个人 API Key 和模型名称。

建议使用课程组提供的网关地址和个人 API Key，允许自行选择符合课程要求的模型。

特别注意：`**.env**` 只用于本地运行环境，不属于需要提交的正式任务文件，禁止将其打包上传。

1. .env 的获取方式：由课程组统一提供

`.env` 中涉及课程网关和个人 API Key，不能自行生成或推导这些信息。

三个配置字段的来源如下：

配置字段	作用	获取方式	是否可以自行修改
`LLM_BASE_URL`	课程 LLM 网关连接地址	由课程组提供	不可以
`LLM_API_KEY`	个人课程 API Key	由课程组按照学号发放	不可以
`LLM_MODEL`	本次使用的模型名称	自行选择并填写	可以

具体要求：

- `LLM_BASE_URL` 必须使用课程组公布的网关地址，不能随意替换为其他模型服务地址。
- `LLM_API_KEY` 必须使用本人对应的课程 Key，不能使用其他人的 Key。
- `LLM_MODEL` 是自主填写的字段，必须填写实际准备使用的模型名称。
- 三个字段均为必填项，不能留空。

特别注意：

发行包中的 `.env.example` 只是配置模板，不包含真实的网关连接信息和 API Key。

安装程序不会自动生成 API Key。必须通过课程组提供的课程平台、邮件或群公告获取真实配置。

2. .env 必须填写哪些内容?

标准配置文件包含以下三个字段，例如：

```
LLM_BASE_URL=https://课程组提供的网关地址
LLM_API_KEY=你的个人课程KEY
LLM_MODEL=qwen3.7-flash
```

其中，`qwen3.7-flash` 仅为模型名称填写示例，实际使用时应根据课程组开放的模型和自己的选择填写

3. .env 应该放在哪里?

这是最容易出现配置错误的地方。

系统要求 `.env` 放在 `workspace/` 根目录下，作为 Phase 1、Phase 2 和 Phase 3 共用的运行配置文件。

正确目录结构：

```
workspace/
|
├── .env
├── .env.example
|
├── phase1/
|   ├── task/
|   ├── agent/
|   └── metadata.json
|
├── phase2/
|   ├── task/
|   ├── agent/
|   └── metadata.json
|
└── phase3/
```

正确位置：

```
workspace/.env
```

以下位置不是当前系统要求的配置位置：

```
workspace/phase1/.env
workspace/phase2/.env
workspace/phase3/.env
```

只需要在 `workspace/` 根目录配置一次，三个阶段共用这一份文件，不需要分别创建三份 `.env` 。

4. 如何创建和填写 .env?

如果发行包中已经提供 `.env.example` ，可以按照以下步骤操作。

第一步：进入 workspace 目录。

```
cd workspace
```

第二步：复制配置模板。

```
cp .env.example .env
```

第三步：编辑 `.env`。

```
nano .env
```

第四步：填写课程组提供的网关地址、个人 API Key 和自己选择的模型名称。

保存后，确认文件存在：

```
ls -la .env
```

最后，检查配置文件中三个字段是否都已填写。

注意：不要把 `.env.example` 直接当作已经完成配置的 `.env`。模板中的字段为空，必须填写真实配置才能正常使用。

5. .env 的格式要求

系统使用简单的 `KEY=VALUE` 格式解析配置文件。

必须遵守以下要求：

- 一个配置项占一行。
- 使用 `=` 分隔字段名称和字段值。
- 字段名称必须与系统规定的名称完全一致。
- 空行可以保留。
- 以 `#` 开头的独立行可以作为注释。
- 字段值两端可以使用单引号或双引号。
- 不允许通过修改字段名称来创建其他配置项。

正确示例：

```
# LLM configuration

LLM_BASE_URL=https://课程组提供的网关地址
LLM_API_KEY=你的个人课程KEY
LLM_MODEL=qwen3.7-flash
```

也可以使用引号：

```
LLM_BASE_URL="课程组提供的网关地址"
LLM_API_KEY="你的个人课程KEY"
LLM_MODEL="qwen3.7-flash"
```

建议不要在配置值后面添加行内注释，也不要使用 `export LLM_MODEL=...` 这种 Shell 环境变量写法。

因为当前配置解析器按照固定的 `KEY=VALUE` 格式读取文件，并不会按照完整 Shell 语法执行。

6. 哪些配置不能由自行修改?

除了三个 LLM 连接字段，系统还设置了统一的运行资源预算。

这些预算由课程运行环境中的 `policy/runtime_policy.json` 管理，不允许通过 `.env` 修改。

配置项目	正式限制
LLM 请求超时	75 秒
LLM 超时重试	最多 1 次超时重试
单次执行累计 Token	10,000 Tokens
Agent 执行时间	90 秒，另有 10 秒启动宽限
Agent 内存	2048 MB
临时磁盘空间	512 MB

特别注意：

以下字段不是当前允许配置的正式项目：

```
LLM_TIMEOUT=120
LLM_TOKEN_BUDGET=20000
```

即使自行添加这些字段，系统也不会因此提高请求超时或 Token 预算。

这些旧配置字段会被忽略并产生警告。

不要尝试通过修改 `**.env**` 绕过课程统一资源限制。

7. 如何检查 .env 是否配置成功?

配置完成后，建议依次检查以下内容：

第一步：检查文件是否存在。

```
test -f .env && echo ".env exists"
```

第二步：检查三个必填字段是否存在且非空。

可以使用以下 Python 命令：

```
python - <<'PY'
```

```
from pathlib import Path

required = [
    "LLM_BASE_URL",
    "LLM_API_KEY",
    "LLM_MODEL"
]

env = {}

for line in Path(".env").read_text(
    encoding="utf-8"
).splitlines():
    line = line.strip()

    if not line or line.startswith("#") or "=" not in line:
        continue

    key, _, value = line.partition("=")
    env[key.strip()] = value.strip().strip("'\"")

for key in required:
    status = "PASS" if env.get(key, "").strip() else "FAIL"
    print(f"{key}: {status}")
PY
```

运行

注意：上述检查只显示配置是否填写，不会打印个人 API Key。

第三步：进行 LLM 连通性测试。

在正式运行 Agent 前，可以使用课程提供的测试程序验证：

- 网关地址是否正确；
- 个人 API Key 是否有效；
- 模型名称是否可用；
- 是否能够正常发送请求并获得模型回复。

仅仅通过 `.env` 文件存在性检查，并不能证明 LLM 一定能够正常连接。

8. .env 是否需要提交?

不需要，而且严禁提交。

`.env` 包含课程网关连接信息和个人 API Key，属于本地运行环境的敏感配置。

系统在打包过程中设置了专门的文件排除和敏感信息检查机制,触发 `FORBIDDEN_FILE` 或 `SECRET_FILE`，导致打包失败。

正常的提交包只包含课程规定的 Task、Agent 和 Metadata 等正式提交材料，不包含本地 LLM 连接配置

9. 提交前必须自查

检查项目	要求
文件位置	必须位于 `workspace/.env`
配置来源	网关地址和个人 API Key 来自课程组
`LLM_BASE_URL`	已填写正确的课程网关地址
`LLM_API_KEY`	已填写本人有效的课程 Key
`LLM_MODEL`	已填写实际使用的模型名称
必填字段	三个字段均非空
配置格式	使用规定的 `KEY=VALUE` 格式
阶段共用	Phase 1、Phase 2、Phase 3 共用根目录配置
连通性	能够正常调用课程 LLM 网关
资源预算	不通过 `.env` 修改系统固定预算
文件安全	不泄露、不上传、不打包 `.env`

最后强调：.env 是本地运行配置，不是正式提交文件。

课程组提供网关地址和个人 API Key，只需要填写模型名称，并在 `workspace/` 根目录完成一次配置。

配置完成后，三个阶段共用同一份 `.env`。正式提交时，系统会自动排除该文件，不需要也不应该将其上传。

c) 本地验证

完成本地测试后，**必须在提交前执行一次正式验证**：

```
bin/datac submission validate \
  --input /workspace/submission/bigdata_p1_<学号>.tar.gz \
  --phase P1
```

这一步是正式提交前的最后一次自检。

因此，只要你本地使用最终准备提交的 ZIP 执行：

```
local_validation = PASS
```

那么助教收到**同一个 ZIP 文件**后，在课程环境一致的情况下，通常也会得到相同的验证结果。

所以，不要等提交以后才让助教帮你发现问题。提交前，自己就可以先发现：

```
Task 文件是否缺失
Data 文件是否正确
reference.py 是否能运行
Agent 是否能运行
学号是否一致
提交阶段是否正确
文件结构是否符合要求
必填字段是否完整
```

系统会检查提交内容并生成课程要求的最终提交文件。

不要手工修改 ZIP，也不要自己重新压缩来绕过检查。

请务必验证“最终要提交的那个 ZIP”，不要只验证工作区。助教收到的也是这个 ZIP，助教侧会用同一套验证程序重新检查。

3.2 理解 Agent 运行结果

本地测试结果可以先分成三类。

情况一：Agent 正确完成任务

Agent 正常执行，并且最终输出与标准答案一致。

重点不是只看“运行成功”，还要确认与reference.py执行结果：

```
Agent Output == Ground Truth
```

Agent 执行结果：Agent Output

reference.py执行结果：Ground Truth

情况二：Agent 运行成功但答案错误

Agent 程序正常运行，但最终答案错误。

此时重点分析：

- 是否理解错题目；
- 是否选择错字段；
- 是否漏掉过滤条件；
- 是否出现数据类型转换错误；
- 是否使用了错误的统计方法；
- 是否输出格式不符合要求。

情况三：Agent 运行失败

程序或模型调用没有正常完成。

常见检查方向：

- Python 异常；
- 文件路径错误；
- 数据文件不存在；
- API 调用失败；
- Agent 接口不符合模板；
- 输出没有正常返回；
- Docker / 本地课程服务没有启动。

第四部分：阶段 2 — 匿名任务挑战与优化

4.1 任务一：匿名Tasks分析BigData

阶段二收到的是其他同学设计的匿名 Task。

> 通过真实 Task 和真实 Agent 运行结果，分析数据分析任务设计与 Agent 能力边界。

假设老助教发放下来的包是：

```
BigData_Exposure_R000001.tar.gz
```

导入匿名Tasks

```
bin/datac exposure import \
  --package /workspace/BigData_Exposure_R000001.tar.gz \
  --workspace /workspace
```

检查：

```
bin/datac exposure status \
  --workspace /workspace
```

目标：

```
tasks=10/10
done=0
```

进入workspace工作区,查看exposure文件夹

此时会看到ATxxx（示例条目），强调每个AT XXX对应的是一个匿名Tasks

测试匿名 Task

用自己的 Agent v1 跑 10 个匿名任务

先确认 `.env` 存在：

```
ls -lh workspace/.env
```

测试发放的 10 个匿名任务：

```
bin/datac exposure run \
  --workspace /workspace
```

跑完检查：

```
bin/datac exposure status \
  --workspace /workspace
```

目标：

```
[exposure] status=READY tasks=10/10 done=10
```

分析匿名 Task

1.题目要求是否清楚?

检查:

- 目标是否明确;
- 是否存在多种解释;
- 是否明确要求计算什么;
- 是否可以判断最终答案类型。

2.数据是否支持问题?

检查:

- 所需字段是否存在;
- 数据是否足够;
- 字段含义是否清楚;
- 是否存在无法从数据中确定的信息。

最终要求填写review-comments.json

⭐ ⭐ ⭐ 评审意见文件填写 ⭐ ⭐ ⭐

`review-comments.json` 是对系统分配的匿名 Task 提交评审意见的文件。

需要使用自己编写的 Agent，对系统分配的匿名任务进行测试，并结合实际运行结果，评价任务的质量、难度及存在的问题，填写相应评分和文字意见。

1. 文件格式与基本结构

系统要求 `review-comments.json` 必须是合法的 JSON 文件，顶层使用对象（字典）结构。

基本格式如下:

```
{
  "AT005": {
    "task_id": "AT005",
    "score": 3,
    "comments": "填写对该任务的实际评审意见。"
  },
  "AT006": {
    "task_id": "AT006",
    "score": 4,
    "comments": "填写对该任务的实际评审意见。"
  }
}
```

以上仅展示 JSON 格式，评分是示例值，不代表对应匿名任务的真实质量。正式提交时必须填写全部分配任务，并根据实际测试情况评分。

其中：

- `AT005`、`AT006`：系统分配的匿名任务编号。
- `task_id`：当前评审任务的匿名编号。
- `score`：对该任务的评分，必须是 1～5 的整数。
- `comments`：对任务的具体评审意见，必须是非空字符串。

特别注意：

每条评审记录必须且只能包含 `task_id`、`score`、`comments` 三个字段，不允许增加其他字段。

```
{
  "ATxxx（示例条目，请整条替换）": {
    "task_id": "ATxxx（示例条目，请整条替换）",
    "score": 0,
    "comments": "模板条目：每道分配给你的题各写一条；score 为 1~5 的整数；comments 为非空评语。导入正式分配后请整条替换本示例。"
  }
}
```

填写内容严格按照每个AT XXX对应进行填写，否则最终阶段是无法正常导出提交包

2. 必须评审系统分配的全部任务

需要按照系统提供的任务分配记录填写评审意见。

具体要求：

- 每个被分配的匿名 Task 都必须有一条评审记录。
- 不能遗漏系统分配的任务。
- 不能自行增加未分配的任务。
- 不能重复填写相同的匿名任务编号。
- 顶层编号必须与对应的 `task_id` 一致。

例如，系统分配了 `AT005`、`AT006`、`AT007` 三个任务，就必须分别填写这三个任务的评审意见。

不能只提交其中两个，也不能额外加入 `AT020`。

评审数量以系统实际分配结果为准，不要自行假设固定数量。

当前 Exposure 分配示例为 10 个匿名任务，因此对应的评审文件需要包含 10 条记录。

3. task_id：匿名任务编号

`task_id` 表示当前正在评审的匿名任务。

要求：

- 必须使用系统分配的真实匿名编号。
- 必须与该条记录的顶层 Key 对应。

例如：

```
{
  "AT005": {
    "task_id": "AT005",
    "score": 3,
    "comments": "评审意见"
  }
}
```

这是符合编号对应关系的写法。

如果顶层写 `AT005`，但内部 `task_id` 填写 `AT006`，系统会认为编号不一致，拒绝提交。

4. score：任务评分

`score` 必须是 1～5 之间的整数。

填写示例	是否符合格式
`1`	符合
`3`	符合
`5`	符合
`0`	不符合
`6`	不符合
`3.0`	不符合
`"3"`	不符合
`true`	不符合

特别注意：

- 不能填写小数。
- 不能把数字写成字符串。
- 不能填写布尔值。
- 不能超出 1～5 的范围。

评分必须有实际依据。

应根据分配任务的题目要求、数据情况、Agent 测试结果和发现的问题进行评价。

不能在没有测试和分析的情况下，给所有任务随意填写相同的分数。

需要说明的是，1～5 分的具体质量等级和评分标准，应以课程正式发布的评分细则为准，不能自行将分数解释为系统未规定的等级。

5. comments：具体评审意见

`comments` 必须是非空字符串。

不能填写空字符串、纯空格或数字等非字符串内容。

例如：

```
"comments": ""
```

或者：

```
"comments": "   "
```

均不符合要求。

但需要特别注意：

系统能够通过格式检查，不代表评审意见具有实际价值。

例如：

```
"comments": "1"
```

虽然属于非空字符串，可以通过基础格式校验，但没有说明任何评审依据，不符合课程要求的实质性评审目的。

建议在 `comments` 中说明以下内容：

① **任务要求：** 简要说明该任务要求 Agent 完成什么计算。

② **测试结果：** 说明自己的 Agent 是否完成任务，以及实际输出情况。

③ **问题分析：** 如果 Agent 答错，说明可能在哪个步骤出现问题；如果任务本身存在歧义或信息不足，应指出具体位置。

④ **评价依据：** 解释为什么给出相应评分，不能只写“很好”“太难了”“模型没答对”。

例如，一道任务要求先选择每个用户的最新记录，再判断其是否为 ACTIVE。

如果 Agent 错误地先筛选 ACTIVE，再选择最新记录，可以在评审意见中说明这一现象，以及它为什么会影响答案。

需要区分：

- Agent 没有正确解题，不一定说明 Task 本身有问题。
- Agent 正确解题，也不代表 Task 一定具有足够挑战性。
- Task 存在规则歧义或数据缺失，需要单独指出。
- 如果没有足够证据确认问题，应如实说明，不能猜测或编造测试结果。

6. 必须遵守匿名评审原则

系统分配的 Task 使用匿名编号，例如 `AT005`。

只能根据任务本身的内容和实际测试结果进行评价。

具体要求：

- 不得在评审意见中讨论作者和自己个人信息。
- 不得通过联系其他获取参考答案或私有材料。
- 不得擅自修改收到的匿名 Task。
- 不得将自己的推测当作已验证的事实。
- 如果任务信息缺失、题目存在歧义或无法判断，应在 `comments` 中说明。

匿名评审的目的是评价 Task 本身，而不是评价出题作者。

7. 只能使用系统分配的评审材料

Exposure 提供的匿名任务材料不包含作者的 Agent 代码、参考答案或其他私有材料。

应当根据实际收到的任务内容，结合自己 Agent 的运行结果完成评价。

不允许：

- 通过外部渠道寻找匿名任务的作者。
- 获取未授权的 Reference 或 Gold。
- 修改匿名任务的数据以改变测试结果。
- 通过猜测标准答案代替实际测试。
- 在没有证据的情况下编造任务缺陷。

如果任务无法运行，或者由于信息不足无法确认正确结果，应如实记录实际情况。

8. 文件提交与打包要求

当前系统使用统一的 `review-comments.json` 文件提交评审意见。

以前使用多个独立评审文件的 `reviews/` 提交方式已经取消。

因此：

- 所有匿名任务的评分和意见都必须写在同一个 `review-comments.json` 文件中。
- 不要把每一道任务拆分成独立 JSON 文件。
- 不要自行修改文件名。

- 不要将旧版 `reviews/` 目录作为正式提交内容。
- 不要在 JSON 中添加注释或其他不符合标准 JSON 语法的内容。

此外，课程要求单独提交实验报告，文件名为：

```
实验报告.docx
```

实验报告应根据课程提供的 `实验报告模板.docx` 编写。

注意：模板文件本身不是正式提交的实验报告。

9. 提交前必须自查

检查项目	要求
文件名称	`review-comments.json`
JSON 格式	能够正常解析
顶层结构	必须是 JSON 对象
任务数量	与系统实际分配任务一致
任务编号	不能缺失、重复或增加
每条记录字段	只能包含 `task_id`、`score`、`comments`
task_id	与对应的顶层任务编号一致
score	必须是 1～5 的整数
comments	必须是非空字符串
测试依据	基于自己的 Agent 的实际测试情况
评审内容	说明任务表现、发现的问题及评分依据
匿名要求	不推测或披露作者身份
提交方式	所有评审集中在一个 JSON 文件中

最后强调：review-comments.json 不是简单填写几个分数，而是要求根据实际测试结果，对分配的匿名任务进行评价。

系统会检查文件结构、任务编号、评分类型和评审内容是否填写完整，但这些格式检查不能代替对评审质量的判断。

必须认真运行任务、分析结果并填写有依据的评审意见，不能通过随意打分或填写无意义文本应付评审要求。

任务二：Task-Agent 最终方案

敲定Task 方案

Phase 2 不另外创建 `task_v2.json`。

仍然修改原来的：

```
workspace/task.json
workspace/data.csv
```

例如直接编辑：

```
nano workspace/task.json
```

如果 Task v2 需要调整数据：

```
nano workspace/data.csv
```

也可以直接用 VS Code 修改。

敲定Agent 方案

Phase 2 不另外创建 `agent_v2.py`。

仍然修改：

```
workspace/agent.py
```

例如直接编辑：

```
nano workspace/agent.py
```

逻辑就是：

```
Agent v1
   ↓
跑 10 个陌生任务
   ↓
发现失败原因并且分析
   ↓
修改 agent.py
   ↓
Agent v2
```

测试 Task-Agent 最终方案

本地运行测试指令:

```
bin/datac run --workspace /workspace --phase P2
```

它对应:

```
task.json + data.csv
        ↓
agent.py
        ↓
本地 Runtime 执行
        ↓
生成运行结果
```

4 构建 Phase 2 提交包

在课程目录执行:

```
bin/datac build --phase P2 --workspace /workspace --student-id <学号>
```

目标:

```
[submission] build status=built
local_validation=PASS
error_code=None
```

最终得到：

```
/workspace/phase2/submission/bigdata_p1_20260001.tar.gz
```

也就是：

```
bigdata_p1_<student-id>.tar.gz
```

本地验证

本地测试完成后执行：

```
bin/datac submission validate --input /workspace/phase2/submission/bigdata_p1_<学号>.tar.gz
```

系统会检查提交内容并生成课程要求的最终提交文件。

不要手工修改 ZIP，也不要自己重新压缩来绕过检查。

请务必验证“最终要提交的那个 ZIP”，不要只验证工作区。助教收到的也是这个 ZIP，助教侧会用同一套验证程序重新检查。

任务三：实验报告内容要求

实验报告不要求大量展示代码和运行日志，重点说明 Task 是怎么设计的、Agent 是怎么设计的、从 v1 到 v2 为什么改或为什么不改，

以及通过本次实验获得了哪些经验和后续优化方向。

系统对实验报告有以下硬性要求：

检查项目	具体要求
文件名称	必须为 `实验报告.docx`
保存位置	`workspace/phase2/实验报告.docx`
文件格式	必须是符合基本 DOCX 结构的文件
文件大小	不能为 0 字节

检查项目	具体要求
模板使用	基于课程提供的实验报告模板填写
旧文件名	`实验报告模板.docx` 不能作为正式提交文件名
打包要求	实验报告必须包含在 Phase 2 提交包中

特别注意：

- 必须使用 `.docx` 格式，不能直接提交 `.doc`、`.pdf` 或 `.txt`。
- 不能仅将其他格式文件的后缀修改为 `.docx`。
- 建议使用 Microsoft Word 或其他兼容软件编辑，并确保文件能够正常打开。
- 原始课程模板是填写依据，不应将未填写的模板直接作为实验报告提交。
- 正式提交时，文件名必须准确写为 `实验报告.docx`。

2. 实验报告必须填写哪些内容？

实验报告按照课程模板，主要包括以下三个部分：

一、Task 设计与迭代

这一部分只需要说明**你的 Task 是怎么设计的，以及从 v1 到 v2 有没有变化**。

1. Task v1 设计思路

说明你第一阶段设计的 Task。

重点回答：

- 这个 Task 要解决什么数据分析问题；
- 为什么选择这个问题；
- 使用了什么数据；
- 主要考察哪一种或哪几种数据分析能力；
- 你希望这个 Task 能够暴露 Agent 的什么能力边界。

不需要大篇幅粘贴代码，也不需要逐字段解释 `task.json`。

可以围绕：

> **问题是什么 → 为什么这样设计 → 想考察什么能力**

来写。

2. Task v2 设计思路

说明第二阶段最终提交的 Task。

重点回答：

- Task v2 最终是怎样设计的；
- 与第一阶段相比，设计目标是否发生变化；
- 为什么最终形成现在这个版本。

如果 Task v2 与 Task v1 完全相同，也可以。

此时必须明确写：

> **Task v2 与 Task v1 保持一致，本阶段未对 Task 进行修改。**

并简单说明为什么没有修改。

3. Task v1 → Task v2 的改变

如果进行了修改，直接说明：

```
Task v1
   ↓
发现的问题 / 得到的启发
   ↓
修改内容
   ↓
Task v2
```

例如可以写：

项目	Task v1	Task v2	修改原因
问题设计	原设计	修改后设计	为什么修改
数据设计	原数据特点	修改后特点	为什么修改
难点设计	原能力点	新能力点	为什么修改

重点不是“改了多少”，而是：

> **为什么改。**

如果没有修改，则直接写：

> 经第二阶段测试与分析后，未发现当前 Task 存在必须调整的问题，因此 Task v2 保留 Task v1 的设计。

二、Agent 设计与迭代

这一部分同样只讲：

> **Agent 是怎么设计的，以及从 v1 到 v2 有没有改变。**

1. Agent v1 设计思路

说明第一阶段 Agent 的总体设计。

重点回答：

- Agent 如何理解 Task；
- Agent 如何使用 CSV 数据；

- Agent 如何进行数据分析；
- 是否使用 LLM；
- 大致的解题流程是什么；
- 你设计这个 Agent 时最关注什么能力。

例如可以概括成：

```
读取 Task
   ↓
理解题目要求
   ↓
分析 CSV Schema
   ↓
规划分析步骤
   ↓
执行数据分析
   ↓
检查结果
   ↓
输出答案
```

不需要把整个 `agent.py` 代码粘贴到实验报告。

实验报告要说明的是：

> **你的 Agent 为什么这样设计。**

2. Agent v2 设计思路

说明第二阶段最终 Agent 的设计。

重点回答：

- Agent v2 最终采用什么思路；
- 是否针对 Phase 2 中观察到的问题做了调整；
- 最终希望解决 Agent v1 的什么不足。

如果没有修改，也完全允许。

此时必须明确写：

> **Agent v2 与 Agent v1 保持一致，本阶段未对 Agent 进行修改。**

同时简单说明你的判断理由。

3. Agent v1 → Agent v2 的改变

如果进行了修改，重点说明：

```
Agent v1 出现什么问题
        ↓
你如何判断问题原因
        ↓
采取什么修改
        ↓
希望 Agent v2 改善什么
```

例如：

发现的问题	Agent v1	Agent v2	改进目的
Schema 理解不稳定	原策略	新策略	提高字段理解
缺少结果验证	直接输出	增加检查	降低错误答案
分析规划不足	一次完成	分步骤分析	提高复杂任务稳定性

这里也不要求一定修改。

如果没有修改，可以直接说明：

> 根据第二阶段的运行与分析结果，没有发现足够明确的问题需要修改 Agent，因此最终 Agent v2 保留 Agent v1 的实现。

三、总结与反思

最后只需要进行总体总结，不再增加新的技术章节。

重点回答三个问题：

1. 这次实验你学到了什么？

例如：

- 对数据分析 Task 设计有什么新的认识；
- 对 Agent 能力边界有什么认识；
- 什么样的问题容易让 Agent 失败；
- 如何判断一个 Agent 是否真正具有通用数据分析能力。

2. 哪些设计经验以后可以继续使用？

例如：

```
Task 要有清晰目标
Reference 要可靠
Agent 要先理解 Schema
复杂任务需要分解
结果必须验证
```

重点总结**可迁移的经验**。

3. 如果继续迭代，下一步准备优化什么？

这里写未来方向即可，例如：

- 提高 Agent 对复杂 Schema 的理解；
- 增强 Join 能力；
- 增加结果验证机制；
- 改进任务规划；
- 设计更有区分度的 Task；
- 增加边界数据和异常数据；
- 提高 Agent 的稳定性和 Token 使用效率。

最终实验报告结构

因此整份 Word 实验报告可以非常清楚地压缩成：

```
1. Task 设计与迭代
   1.1 Task v1 设计思路
   1.2 Task v2 设计思路
   1.3 Task v1 → Task v2 的改变
       （若无改变，明确写“未修改”并说明原因）

2. Agent 设计与迭代
   2.1 Agent v1 设计思路
   2.2 Agent v2 设计思路
   2.3 Agent v1 → Agent v2 的改变
       （若无改变，明确写“未修改”并说明原因）

3. 总结与反思
   3.1 实验收获
   3.2 可复用的设计经验
   3.3 后续优化方向
```

另外，**Task v2 / Agent v2 不强制必须发生修改**。如果第二阶段与第一阶段完全一致，也是允许的，但必须在实验报告相应位置明确写明“未修改，与 v1 保持一致”，并说明判断依据。

第五部分：常见问题与提交前检查

5.1 常见问题排查

Q1：Docker 服务没有启动怎么办？

```
systemctl start docker.service
systemctl status docker --no-pager
```

Q2：`docker compose` 不存在怎么办？

先执行：

```
docker compose version
```

如果不存在，再检查：

```
docker-compose version
```

如果旧版 `docker-compose` 可以正常输出，则说明当前安装的是 Compose v1。课程发布包如果要求 Compose v2，应优先按照前文安装 Compose v2。

Q3：`run` 失败怎么办？

如果是阶段一执行run失败优先检查：

```
ls -lah workspace/phase1
python workspace/phase1/reference.py
```

然后确认：

- `task.json` 是否存在；
- 数据文件名是否正确；
- `reference.py` 是否可以独立运行；
- `agent.py` 是否符合课程接口；
- API 配置是否正确；
- 课程服务是否启动。

阶段二则同理

Q4：Agent 能运行，但是答案错误怎么办？

这通常不是“环境安装问题”，而是数据分析逻辑问题。

按顺序检查：

```
1. Task 是否理解正确
2. 字段是否选择正确
3. 数据类型是否正确
4. 过滤条件是否完整
5. 分组 / 聚合是否正确
6. 最终输出类型和格式是否正确
```

5.2 快速操作参考

系统推荐的实践流程

特别注意：程序能够运行、答案正确和提交包校验通过，是三个不同的检查环节。 必须分别完成，不能只看到某一步显示 PASS，就认为整个项目已经符合要求。

1. 阶段 0：安装环境与系统自检

首次使用系统时，需要完成运行环境安装与检查。

依次执行：

```
bash scripts/install.sh
bin/datac doctor --workspace /workspac
```

其中：

- `install.sh`：加载 Docker 镜像并启动课程运行容器，安装过程采用离线方式。
- `datac doctor`：检查 Python 环境、Runtime Policy 和必要的环境变量配置。

正常情况下，系统应能够完成环境检查并显示 `PASS`。

--workspace /workspace 是容器内路径（宿主机路径是 ./workspace）

特别注意：

- `.env` 文件放在 `workspace/` 根目录，不是 `phase1/` 或 `phase2/` 目录。
- Phase 1 和 Phase 2 共用同一份 `.env` 配置。
- 需要按照课程要求配置 `LLM_BASE_URL`、`LLM_API_KEY` 和 `LLM_MODEL`。
- 不得通过修改 `.env` 绕过系统的运行时间、内存、Token 等资源限制。
- `.env` 中可能包含 API Key，不得上传到公共仓库或加入最终提交包。

建议先确认 `**doctor**` 检查通过，再进行后续 Task 和 Agent 开发。

2. 阶段 1：完成 Phase 1（Task v1 + Agent v1）

Phase 1 的主要工作是设计第一版 Task，并编写能够解决未知数据分析任务的 Agent。

第一步：完成 Task 与 Agent 文件

需要按照前述规范填写：

文件	编写要求
`task.json`	明确题目描述、计算规则和标准答案
`data.csv`	提供完整、合法的题目数据
`reference.py`	正确实现题目要求，返回计算结果
`agent.py`	定义 `run(task, df, llm)`，完成智能体求解
`metadata.json`	填写学号、Task ID、Agent ID 和失败模式等信息

根据当前冻结版本：

- `task.json` 只保留 `description`、`task_statement`、`answer` 三个字段。

- `hypothesized_failure_mode` 填写在 `metadata.json`。
- 失败模式必须使用完整的规范名称，例如 `FM_AGGREGATION`，不能写成 `FM06_AGGREGATION`。

第二步：运行 Reference，检查标准答案

```
bin/datac reference run --phase P1
```

系统会执行 `reference.py`，检查程序能否正常计算。

预期结果：

```
Status: PASS
{"answer": 30}
```

其中 `30` 仅为示例，实际结果以设计的 Task 为准。

这一步不需要运行 Agent，也不消耗 LLM Token。

特别注意：Reference 运行 PASS，只说明相应运行检查通过，不能单独证明程序完全符合题意。

第三步：运行自己的 Agent

```
bin/datac run --workspace /workspace --phase P1
```

这一步会真正运行 Agent，并调用系统配置的大语言模型。

运行结束后，应查看：

```
workspace/runs/phase1/<Run_ID>/result.json
```

重点检查：

- Agent 是否正常完成执行；
- 是否正确返回最终答案；
- 返回结果是否与标准答案一致；
- 是否存在模型接口错误或资源超限。

建议将 `correct=True` 作为 Phase 1 的基本自测目标。

但需要注意，自己的 Agent 能解答自己设计的 Task，不代表它能够解答其他人的未知任务。

第四步：生成并校验 Phase 1 提交包

当前冻结版本推荐使用：

```
bin/datac build --phase P1 --student-id <学号>
```

生成文件：

```
/workspace/store/packages/bigdata_p1_<学号>.tar.gz
```

Phase 1 提交包顶层严格包含系统规定的 3 个文件，不得自行添加文件。

生成后还应按照课程提供的验证命令检查提交包是否合规，确认构建及校验均通过。

3. 阶段 2：完成 Phase 2（Task v2 + Agent v2 + 评审材料）

Phase 2 需要在现有项目基础上继续修改，不是重新创建一套无关项目。

第一步：修改 Phase 2 文件

直接修改 `phase2/` 中对应的文件：

- `task.json`
- `data.csv`
- `reference.py`
- `agent.py`

特别注意：

- 不要另外创建 `task_v2.json`、`reference_v2.py` 等文件。
- Phase 2 的 `task_id`、`agent_id` 必须与 Phase 1 保持一致。
- 如果修改了数据或参考程序，必须重新检查 `answer`。
- 不能为了提高难度而故意使题干、数据与标准答案不一致。

第二步：核对 Phase 2 标准答案

```
bin/datac reference run --phase phase2
```

确认 Reference 能正常执行，且返回结果与 `task.json.answer` 一致。

第三步：运行 Phase 2 Agent

```
bin/datac run --workspace /workspace --phase P2
```

查看结果：

```
workspace/runs/phase2/<Run_ID>/result.json
```

建议确认运行正常、`correct=True`，同时检查资源使用与异常日志。

第四步：完成评审材料

Phase 2 除代码文件外，还需要完成以下材料。

（1）review-comments.json

根据系统分配的 AT 编号填写评审意见。

要求：

- 每个分配的 `AT###` 必须恰好有一条评审记录。
- `score` 必须在 1—5 的允许范围内。
- `comments` 必须包含具体、有意义的评审内容。
- 评审意见应结合实际 Task 的题目要求、数据和执行情况。
- 不能使用 `"1"`、`"通过"` 等缺乏实质内容的文字代替分析。

需要注意：机器校验通过，不代表评审意见质量合格。

（2）实验报告.docx

按照课程提供的报告模板填写。

要求：

- 补充姓名、学号、班级等必要信息。
- 填写实验设计、实现方法、运行过程、实验结果与分析。
- 根据真实实验记录说明遇到的问题和改进情况。
- 删除或填写所有 `【填写】` 占位符。
- 按要求保留规定的文件名和文档格式。

不得保留空白模板作为正式实验报告，也不得编造未执行的实验结果。

第五步：生成 Phase 2 提交包

```
bin/datac build --phase P2 --workspace /workspace --student-id <学号>
```

生成文件：

```
workspace/store/packages/bigdata_p2_<学号>.tar.gz
```

Phase 2 提交包顶层严格包含系统规定的 5 个文件。

随后执行：

```
bin/datac submission validate --input workspace/store/packages/bigdata_p2_<学号>.tar.gz
```

确认提交包校验通过。

4. 推荐的开发与修改顺序

建议按照以下顺序反复检查：

设计 Task → 运行 Reference → 运行 Agent → 检查结果 → 修改问题 → 重新测试 → 生成提交包。

不同运行结果应采取不同的处理方式。

运行结果	含义	建议操作
`completed`	正常完成且答案正确	保留结果，继续下一阶段
`analysis_failure`	正常返回答案，但与 Gold 不一致	分析解题错误，检查 Task、Reference 与 Agent
`internal_error`	模型接口、资源限制或程序异常	排查接口、环境、程序和资源使用

特别注意：

不要把 Agent 算错题和程序运行失败混为一谈。

如果是 `analysis_failure`，应重点分析 Agent 为什么没有正确理解或完成题目，而不是直接修改标准答案。

如果是 `internal_error`，应根据具体错误类别排查原因，不能直接判断为 Task 设计错误。

同时，不能仅为了让自己的 Agent 返回 `correct=True`，随意降低题目难度或修改 Gold。

正式提交统一使用 `datac build --phase P1/P2` 生成 TAR.GZ 文件。

不要自行改变工作区结构，也不要为了使用旧版 ZIP 打包命令而复制、移动或重新组织提交文件。

6. 提交前必须自查

`datac doctor` 检查通过。

`.env` 位于 workspace 根目录，模型配置完整。

`metadata.json` 中学号、Task ID、Agent ID 填写正确。

Phase 1 和 Phase 2 的 Task ID、Agent ID 保持一致。

`hypothesized_failure_mode` 使用规范的 Failure Mode Key。

`task.json` 使用当前冻结版本规定的三个字段。

`data.csv` 满足文件大小、行数、列数和编码要求。

`reference.py` 正确实现题意，`reference run` 为 PASS。

`agent.py` 符合接口要求，并已进行实际运行测试。

`task.json.answer` 与正确计算得到的结果一致。

`review-comments.json` 已按分配的 AT 编号填写实质性评语。

`实验报告.docx` 已填写完整，无未处理的占位符。

正式提交包通过 `datac build` 构建。

提交包完成相应校验并通过。

提交包不包含 `.env`、`runs`、`exposure`、`store`、`__pycache__` 等非提交文件。

最后强调：需要提交的不只是能够运行的程序，还包括清楚、可验证的 Task，能够独立求解的 Agent，以及真实、完整的实验与评审材料。

系统检查通过是提交的基本条件，但不能代替对 Task 质量、Agent 能力和评审内容的实际判断。

附页A｜正式评测：精选测试集 + 攻防对抗

> **两种模式同时计入最终成绩**：精选测试集考察 Agent 的综合解题能力；攻防对抗考察 Agent 进攻表现与 Task 防守能力。

① 提交什么：最终参评成果

成果	正式评测使用规则
Agent v2（矛）	**唯一**参加最终智能体评测；Agent v1 仅用于过程测试与迭代分析。
Task v1 + Task v2（盾）	两道题**分别审核**，合格的题目都有机会参与评测，最多 2 道。

② 什么有资格参评：规范与质量审核

Task 须题意明确、没有歧义、参考程序与标准答案正确，且自己的 Agent 能成功求解；应针对课程规定的 Failure Mode 设计。Agent 也须通过运行与提交规范审核。**超时、资源超限和运行时错误不能作为预期 Failure Mode 的有效证明。** 未通过审核的成果不具备对应参评资格，因此实际可能是 **0～1 个矛、0～2 个盾**。

③ 怎么评测：两种模式同时进行

模式一：精选测试集评测	模式二：攻防对抗评测（Elo Rating）
从有效 Task 中按失败模式覆盖、原创性、区分度等筛选优质题目；全部合格 **Agent v2** 统一解题。	**Agent v2 为矛**，挑战其他同学的有效 Task；**Task v1/v2 为盾**，接受其他 Agent 挑战。
Agent 解对的精选题越多，表现越好；**题目入选精选测试集可获得额外加分**。	使用 **Elo Rating** 分别形成进攻与防守排名，并据此计分。

④ 如何计分：两种模式均影响最终成绩

精选测试集成绩体现 Agent 解题能力及入选任务贡献；攻防对抗成绩体现 Agent 进攻与 Task 防守排名。**具体成绩权重、Elo 配对数量、排名换分规则以后续正式评分细则为准**，此处不预设数值。

必须记住的 4 条规则：① 两轮 Task 分别审核、都可能参评；② 最终只评 Agent v2；③ Task 必须自己 Agent 能解；④ 程序崩溃、超时及资源超限不算规定失败模式的有效证明。

模型说明：正式评测计划采用 DeepSeek Flash（暂定 `deepseek-v4.1-flash`）及可能的其他未公开基座模型。Agent 应具备跨模型适应能力，不能针对单一模型结果硬编码。

附页B｜最终成绩：由 4 项评测结果综合计算

> **成绩同时看出题质量与解题能力**：题目质量、精选测试集解题表现、Elo 进攻排名、Elo 防守排名四项结果共同决定最终成绩。

成绩组成	评什么	如何得到分数
① **题目质量得分（(S_Q)）**	Task v1 / Task v2 的有效性、原创性与挑战性	通过任务质量审核；高质量题目入选精选测试集可获得额外加分。
② **测试集解题得分（(S_T)）**	Agent v2 对精选测试集的综合解题能力	Agent v2 统一解题，正确解决的有效题目越多，表现越好。
③ **Elo 进攻得分（(S_A)）**	Agent v2 作为“矛”，挑战其他同学的有效 Task	根据攻防结果计算 Elo 进攻排名，再按公布的规则转换成分数。
④ **Elo 防守得分（(S_D)）**	Task v1 / v2 作为“盾”，接受其他同学 Agent 的挑战	根据攻防结果计算 Elo 防守排名，再按公布的规则转换成分数。

最终成绩计算关系（建议表达，非已确定公式）

最终成绩计算关系

同时保留了加权成绩关系：

$$S_{\text{final}} = w_Q S_Q + w_T S_T + w_A S_A + w_D S_D$$

同时保留了加权成绩关系：**W为对应权重。**

S_Q为题目质量得分，S_T为精选测试集解题得分, S_T为精选测试集解题得分, S_A为Elo进攻得分, S_D为Elo防守得分。

特别说明： 文档明确将此公式标记为建议表达，而非已经确定的正式评分公式。四项权重、精选题入选加分和 Elo 排名换分规则均留待课程组正式公布。

重要说明：这是用于说明成绩构成的**建议加权表达式**，不是已经公布的正式计分公式。四项权重、题目入选加分标准、无效成果处理方式、Elo 对抗数量和排名换分方法，均以课程组后续发布的正式评分细则为准。

课堂强调：① Task v1/v2 与 Agent v2 评价对象不同；② Elo 进攻与 Elo 防守是两套独立排名；③ 只提高 Agent 答题正确率，不等于一定获得最高总成绩。

附页C｜系统实际校验对应的错误码

为方便排查提交失败原因，特此补充：校验失败时应该检查什么？

错误码	触发条件
`CSV_FILE_TOO_LARGE`	文件超过 1,048,576 字节
`CSV_TOO_MANY_ROWS`	pandas 读取后数据行数达到 10,000 行
`CSV_TOO_MANY_COLUMNS`	pandas 读取后列数超过 100
`INVALID_CSV_ENCODING`	文件不符合 UTF-8 编码要求
`INVALID_CSV` 或读取失败提示	CSV 无法正常解析

失败提示或错误码	可能原因
行数超限	代码达到或超过 200 行
语法错误	无法通过 Python AST 解析

失败提示或错误码	可能原因
导入模块不合法	导入了 `numpy`、`os`、`json` 等模块
缺少 `reference(df)`	函数未定义、不可调用或参数名称不符合要求
禁止调用 `open()`	尝试自行打开文件
禁止 pandas 外部读取	使用了 `pd.read_csv()` 等文件读取方法
禁止写出数据	使用了 `df.to_csv()` 等写出方法
`REFERENCE_ERROR`	运行异常或函数执行超时
`ANSWER_MISMATCH`	参考程序返回值与 `task.json.answer` 不一致

补充：常见校验错误与打包注意事项

错误码	含义
`MISSING_METADATA`	缺少 metadata 文件
`MALFORMED_JSON`	JSON 格式错误，无法解析
`BAD_METADATA`	字段缺失、多余或取值不符合 Schema
`STUDENT_ID_MISMATCH`	学号与提交包标识不一致
`INVALID_FAILURE_MODE_KEY`	失败模式名称不在合法枚举中
`METADATA_MISSING_HYPOTHESIZED_FAILURE_MODE`	metadata 中缺少失败模式
`TASK_JSON_FORBIDDEN_HYPOTHESIZED_FAILURE_MODE`	task.json 中仍保留已迁移的失败模式字段

附页D｜系统执行进阶说明

阶段一固定在 phase1 跑，最终阶段固定在 phase2 跑

```
bin/datac reference run --phase phase1
#如果更换了题目，强调每个阶段都在自己的工作区内开发、测试和提交，两个阶段互不覆盖。
bin/datac reference run --phase phase2
```

最终结果类似：

```
============================================================
BigData Reference Runner (v10.1.19)
============================================================
Phase: phase1
Status: PASS

{
  "answer": 30
}
============================================================
reference: /workspace/phase1/reference.py
data:      /workspace/phase1/data.csv
```